TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG

HCMUTE

ĐỒ ÁN MÔN HỌC 2

NGHIÊN CỨU VÀ MÔ PHỎNG MÁY TÍNH SAP-1 SỬ DỤNG PHẦN MỀM VIVADO

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN MÁY TÍNH

Sinh viên: **LÊ HUỲNH MINH NHẬT**
MSSV: 22119109
TRẦN LÊ QUỐC KIỆT
MSSV: 22119094

TP. HỒ CHÍ MINH – 09/2025

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



ĐỒ ÁN MÔN HỌC 2

NGHIÊN CỨU VÀ MÔ PHỎNG MÁY TÍNH SAP-1 SỬ DỤNG PHẦN MỀM VIVADO

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **LÊ HUỲNH MINH NHẬT**
MSSV: 22119109
TRẦN LÊ QUỐC KIỆT
MSSV: 22119094

Giáo viên hướng dẫn**: TRẦN THỊ QUỲNH NHƯ**

TP. HỒ CHÍ MINH – 09/2025

BẢN NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI **1**

1.1 GIỚI THIỆU 1

1.2 MỤC TIÊU ĐỀ TÀI 1

1.3 GIỚI HẠN ĐỀ TÀI 2

1.4 PHƯƠNG PHÁP NGHIÊN CỨU 2

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU 2

1.6 BỐ CỤC QUYỂN BÁO CÁO 3

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **4**

2.1 KIẾN TRÚC VON NEUMAN 4

2.2 TỔNG QUAN KIẾN TRÚC SAP-1 5

2.3 CÁC MODULE THÀNH PHẦN 7

2.3.1 Program Counter (Bộ đếm PC) *7*

2.3.2 Input and MAR-Memory Address Register (thanh ghi bộ nhớ địa chỉ) *7*

2.3.3 Random-Access Memory (RAM) *7*

2.3.4 Instruction Register (thanh ghi lệnh) *8*

2.3.5 Controller-Sequencer (bộ sắp xếp) *8*

2.3.6 Accumulator (bộ tích lũy) *8*

2.3.7 Adder-Subtractor *9*

2.3.8 B-Register *9*

2.3.9 Output Resigter *9*

2.3.10 Binary Display (Màn hình nhị phân) *9*

2.4 TẬP LỆNH (INSTRUCTION SET) 10

2.4.1 LDA *10*

2.4.2 ADD *11*

2.4.3 SUB *11*

2.4.4 OUT *12*

2.4.5 HLT *12*

2.4.2 Mã máy *14*

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG **15**

3.1 THIẾT KẾ PHẦN MỀM 15

3.1.1 Clock *15*

3.1.2 Program counter *17*

3.1.3 Register A *19*

3.1.4 Register B *20*

3.1.5 Adder *21*
3.1.6 Bộ nhớ memory *22*
3.1.7 IR-thanh ghi lệnh *25*
3.1.8 Bộ điều khiển *27*
3.1.9 THANH GHI ĐẦU RA (OUTPUT REGISTER) *30*
3.1.10 Kết nối tất cả các module *31*
3.2 BẢNG TRẠNG THÁI TỪNG MODULE 33
3.2.1 Clock *33*
3.2.2 PC *33*
3.2.3 ALU *34*
3.2.4 IR *34*
3.2.5 CONTROLLER *35*
3.2.6. MEMORY *36*
3.2.7 THANH GHI A *39*
3.2.8 THANH GHI OUTPUT *39*
CHƯƠNG 4: KẾT QUẢ **40**
4.1 KẾT QUẢ CHẠY THỬ NGHIỆM CỦA CÁC MODULE 40
4.1.1 CLOCK TEST *40*
4.1.2 PROGRAM COUNTER TEST *41*
4.1.3 MEMORY TEST *41*
4.1.4 IR TEST *42*
4.1.5 CONTROLLER TEST *42*
4.1.6 ALU TEST *43*
4.2 KẾT QUẢ CHẠY CỦA CÁC CỤM MODULE 43
4.2.1 Nhóm các khối tính toán *44*
4.2.2 Nhóm các khối điều khiển *45*
4.2.3 Kết quả tổng quan chung của hệ thống *46*
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **46**
5.1 KẾT LUẬN **46**
5.2 HƯỚNG PHÁT TRIỂN **48**

DANH MỤC HÌNH

Hình 2.1 Kiến trúc của SAP-1
Hình 3.1 Sắp xếp cột chương trình
Hình 3.2 Sơ đồ khối Clock
Hình 3.3 Lưu đồ giải thuật clock
Hình 3.4 Sơ đồ khối Program Counter
Hình 3.5 Lưu đồ giải thuật Program Counter
Hình 3.6 Sơ đồ khối thanh ghi A
Hình 3.7 Lưu đồ của reg_a
Hình 3.8 Sơ đồ khối thanh ghi B
Hình 3.9 Sơ đồ khối ALU
Hình 3.10 Lưu đồ adder
Hình 3.11 Sơ đồ khối Bộ nhớ
Hình 3.12 Sơ đồ khối thanh ghi lệnh
Hình 3.13 Giải thuật module IR
Hình 3.14 Sơ đồ khối Bộ điều khiển
Hình 3.14 Hình chuỗi đếm state trong IR
Hình 3.15 Sơ đồ thanh ghi đầu ra
Hình 3.16 Tổng quan mô hình SAP-1
Hình 4.1 Waveform module clock
Hình 4.2 Waveform module PC
Hình 4.3 Waveform module Memory
Hình 4.3 Waveform module Memory
Hình 4.4 Chương trình kiểm thử Memory
Hình 4.5 Waveform module IR
Hình 4.6 Waveform module Controller
Hình 4.7 Waveform module ALU
Hình 4.8 Waveform cụm module tính toán
Hình 4.9 Waveform cụm module điều khiển
Hình 4.10 Waveform tổng quan
Hình 5.1 Kết quả sau khi chạy (nguồn: karenok.git)

DANH MỤC BẢNG

Bảng 2.1 Mô tả tập lệnh

Bảng 2.2 Bảng ánh xạ Mnemoniac

Bảng 2.3 Địa chỉ và dữ liệu ở RAM

Bảng 2.4 Bảng ánh xạ opcode

Bảng 3.1 Chương trình nạp vào trong RAM

Bảng 3.2 Bảng dịch chương trình

Bảng 3.3 Bảng vùng dữ liệu trong chương trình

Bảng 3.4 Bảng chức năng của local param

Bảng 3.5 Bảng trạng thái module Clock

Bảng 3.6 Trạng thái Module PC

Bảng 3.8 Ví dụ phép tính trong ALU

Bảng 3.9 Chú thích các chân tín hiệu

Bảng 3.10 Bảng trạng thái tổng quát Memory

Bảng 3.11 Bảng trạng thái thanh ghi A

Bảng 3.12 Bảng trạng thái thanh ghi output

CÁC TỪ VIẾT TẮT

SAP - 1	Simple As Possible 1
ISA	Instruction Set Architecture
I/O	Input/Output
CPU	Central Processing Unit
CLK	Clock
Clr	Clear
Rst	reset
PC	Program counter
MAR	Memory Address Register
RAM	Random Access Memory
IR	Instruction Register
ALU	Arithmetic Logic Unit.
Lo	Load out
hlt	halt
Cp	PC increase
Ep	PC Enable
Lm	MAR load
Ro	RAM out
Li	IR load
Ei	IR Enable
La	Load A
Ea	Enable A
Lb	Load b
Su	Sub
Eu	ALU Enable

BẢNG BÁO CÁO TIẾN ĐỘ

TUẦN	NỘI DUNG CẦN LÀM
4	Viết lý thuyết tổng quan và lý thuyết Von Neuman
5	Viết và tính toán lý thuyết hoạt động
6	Tìm hiểu về tập lệnh
7	Thiết kế module PC
8	Thiết kế module Clock
9	Thiết kế module RAM, MAR
10	Thiết kế module IR
11	Thiết kế module Controller
12	Thiết kế module Register A, B
13	Viết các file testbench
14	Kiểm tra và chỉnh sửa
15	Kiểm tra và chỉnh sửa
16	Hoàn thiện và báo cáo

BẢNG PHÂN CHIA CÔNG VIỆC

STT	Nội dung công việc	Nhật	Kiệt
1	Viết code module và testbench Clock		**X**
2	Viết code module và testbench PC		**X**
3	Viết code module và testbench Memory		**X**
4	Viết code module và testbench IR		**X**
5	Viết code module và testbench Controller		**X**
6	Viết code module và testbench Adder	**X**	
7	Viết code module và testbench register A	**X**	
8	Viết code module register output	**X**	
9	Viết code testbench group calculate	**X**	
10	Viết code testbench group controll	**X**	

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu

Máy tính SAP-1 (Simple As Possible – 1) "Máy tính đơn giản nhất có thể phiên bản 1". Đây là một mô hình máy tính đơn giản được đề cập trong sách "Digital Computer Electronics" của tác giả Albert Paul Malvino. Dựa theo kiến trúc máy tính Von Neuman, SAP-1 thể hiện các chức năng tính toán. Các thành phần của máy bao gồm: bộ nhớ, bộ đếm chương trình, bộ giải mã lệnh, các thanh ghi, bộ tính toán ALU, bộ điều khiển. Trong quy mô đồ án 2, nhóm sẽ thực hiện chạy các chương trình gồm: nạp dữ liệu, cộng, trừ, xuất kết quả, dừng chương trình. SAP-1 chỉ là mô hình giải thích cách mà máy tính và các vi xử lý hoạt động. SAP-1 giúp nhóm hiểu sâu hơn về ISA (Instruction Set Architecture). SAP-1 là nền tảng khởi đầu để theo học các mô hình, kiến trúc phức tạp hơn như: SAP-2, ARM, X86, RICS-V,…

1.2 Mục tiêu đề tài

SAP-1 thực hiện các chức năng của một máy tính theo chu trình lệnh:

Fetch (Nạp lệnh)

⬇

Decode (giải mã)

⬇

Excute (thực thi)

SAP-1 sẽ thực hiện các tập lệnh bao gồm:

LDA: nạp vào thanh ghi tích lũy (accumulator)

ADD: cộng dữ liệu vào accumulator

SUB: trừ dữ liệu từ accumulator

OUT: xuất dữ liệu từ Accumulaotr ra thanh ghi xuất (output register)

HLT: Dừng chương trình

1.3 GIỚI HẠN ĐỀ TÀI

Phạm vi của đề tài này chỉ dừng lại ở mức mô phỏng trên phần mềm môi trường ảo, kém xa so với 1 chiếc máy tính thực tế, là mô hình mẫu để nghiên cứu và học tập và sẽ được cải tiến ở các phiên bản sau. Mô hình hiện tại chứa rất nhiều điểm yếu và giới hạn: bộ nhớ nhỏ, lệnh hạn chế, ALU đơn giản, không có I/O phức tạp

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài được kết hợp bởi nhiều phương pháp nghiên cứu:

Phương pháp tổng hợp lý thuyết: nhóm tổng hợp lý thuyết từ các nguồn như: tài liệu trên github, kiến thức về kiến trúc Von Neuman, các kiến thức về điện tử, mạch điện và kỹ thuật số, các kiến thức về lập trình bằng ngôn ngữ mô tả phần cứng Verilog.

Phương pháp mô phỏng: nhóm sử dụng phần mềm Vivado Xilinx để thực hiện mô phỏng , sau đó sẽ viết code testbench để kiểm thử, kiểm thử đơn lẽ từng module và kiểm thử trên nhiều cụm module để kiểm tra độ chính xác, tránh xung đột dữ liệu

Phương pháp tham khảo lý thuyết, nhóm tham khảo lý thuyết từ các chuyên gia như: Ben Eater và chính tác giả của sách “Digital Computer Electronics” tác giả Albert Paul Malvino

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Ở bài nghiên cứu này, nhóm sẽ tập trung vào nghiên cứu cấu trúc và kiến trúc của máy tính SAP-1, cấu tạo quy trình hoạt động các module. Cách mô phỏng từng module bằng ngôn ngữ mô tả phần cứng verilog. Cách mà dữ liệu di chuyển trong kiến trúc

Phạm vi nghiên cứu của đề tài dừng lại ở mức mô phỏng trên phần mềm, mô phỏng ở mức tối giản, đề tài không đi sâu vào nghiên cứu các kiến trúc CPU hiện đại, giới hạn các tính năng như quản lý nguồn, quản lý I/O, quản lý bộ nhớ.

1.6 BỐ CỤC QUYỂN BÁO CÁO

Chương 1: Giới thiệu đề tài máy tính SAP-1

Chương 2: cơ sở lý thuyết kiến trúc máy tính Vonneuman, kiến trúc của máy tính SAP-1, cách mà từng thành phần hoạt động, cách dữ liệu di chuyển trong kiến trúc, giới thiệu về mã máy

Chương 3: Mô phỏng các thành phần máy, code và giải thuật

Chương 4: Đưa ra các bảng trạng thái của từng module và kết quả chạy để đối chiếu, kiểm tra

Chương 5: Đưa ra kết luận và giới thiệu các hướng phát triển tiên tiến hơn cho SAP-1, các bản nâng cấp như: SAP-2, SAP-3. Các kiến trúc phức tạp hơn như: MIPS, RISC-V, RISC32, RISC64.

Chương 2: CƠ SỞ LÝ THUYẾT

2.1 Kiến trúc Von Neuman

Kiến trúc von Neumann - còn được gọi là mô hình von Neumann hoặc kiến trúc Princeton - là kiến trúc máy tính dựa trên mô tả năm 1945 của nhà toán học và vật lý John von Neuman. Là một mô hình thiết kế máy tính dựa trên việc lưu trữ chương trình và dữ liệu trong cùng một bộ nhớ chính, sử dụng một bộ xử lý trung tâm (CPU), bộ điều khiển, bộ nhớ và thiết bị vào/ra kết nối qua một bus hệ thống duy nhất. Với các thành phần chính:

CPU - Bộ xử lý trung tâm: thực hiện các phép toán và logic trong máy tính, bao gồm hai thành phần chính là ALU (Arithmetic Logic Unit) và Control Unit

Memory – Bộ nhớ: lưu trữ dữ liệu và chương trình

Input/Output (I/O - Thiết bị nhập/xuất): Các thiết bị như bàn phím, chuột, màn hình, máy in, và các thiết bị khác để tương tác với máy tính

Bus (Đường dẫn truyền tải): hệ thống các đường dây truyền dữ liệu giữa các thành phần của máy tính

Mối liên hệ giữa kiến trúc von Neuman và SAP-1:

SAP-1 là một mô hình máy tính có các thành phần cơ bản của kiến trúc Von Neumann, bao gồm: CPU, Memory, Bus, I/O devices. Cả hai đều *sử dụng bộ nhớ chung* chứa chương trình vả cả dữ liệu cần xử lý. *Xử lý lệnh tuần tự,* thực hiện các lệnh trong bộ nhớ theo thứ tự mà chúng được lưu trữ, các lệnh được lấy (fetch), giải mã (decode) và thực thi (execute) một cách tuần tự.

Mặc dù SAP-1 được dựa theo kiến trúc von Neuman, tuy nhiên vẫn có một vài điểm khác biệt: *SAP-1 đơn giản hơn* so với các hệ thống máy/CPU sử dụng kiến trúc von Neuman, không có các yếu tố xử lý đa nhiệm, bộ nhớ cache, thanh ghi cờ, cũng như các phương pháp tối ưu hiệu quả.

Nhìn chung SAP-1 vẫn thể hiện rõ cách mà kiến trúc von Neuman hoạt động trong việc lưu trữ dữ liệu và chương trình chung một bộ nhớ và thực hiện các lệnh tuần tự

2.2 Tổng quan kiến trúc SAP-1

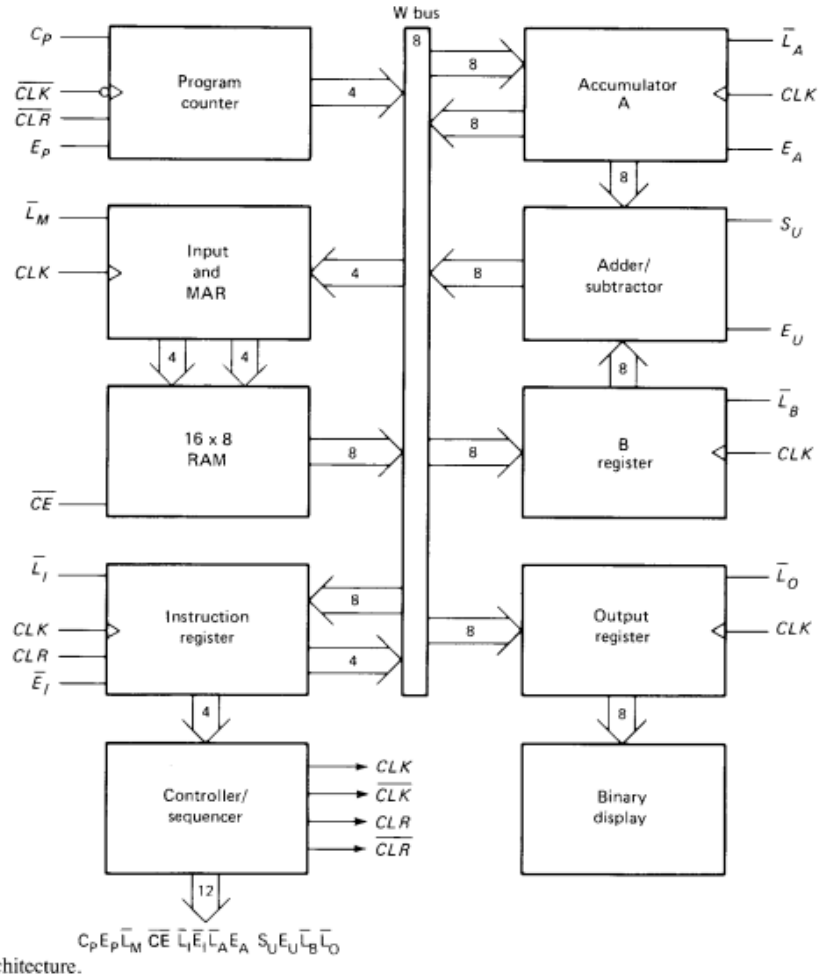


Fig. 10-1 SAP-1 architecture.

Hình 2.1 Kiến trúc của SAP-1

Hình 2.1 mô tả kiến trúc (cấu trúc) của SAP-1, một máy tính tổ chức theo kiểu bus. Tất cả các thanh ghi (register) kết nối với bus W đều có ba trạng thái; điều này cho phép chuyển dữ liệu một cách có trật tự. Tất cả các đầu ra thanh ghi khác đều có hai trạng thái; các đầu ra này liên tục điều khiển các hộp (boxes) mà chúng kết nối.

Máy tính SAP-1 sử dụng bus trung tâm 8 bit và có mười thành phần chính. Hình ảnh minh họa kiến trúc của nó được hiển thị như bên trên. Máy tính SAP-1 của Albert hoạt động như sau:

Mô tả hoạt động:

Máy tính hoạt động theo chu trình fetch, decode và excute. Tương ứng với 6 state từ 0 đến 5. Fetch bao gồm state 0 và 1, decode gồm state 2. Excute gồm state 3,4 và 5. Máy sẽ chạy đồng bộ theo xung clk, tại 1 thời điểm chỉ có 1 chuỗi tín hiệu được phép ở trên bus nhằm tránh xung đột dữ liệu. Việc lấy dữ liệu từ bus hoặc nạp dữ liệu lên bus sẽ do controller kiểm soát bằng cách kích hoạt các chân tín hiệu của từng module. Ví dụ: để PC có thể đếm lên thì tín hiệu Cp (counter program) phải được bật lên 1 để PC bắt đầu đếm, để PC có thể tải dữ liệu lên bus thì tín hiệu Ep (Enable program) phải được kích lên 1.

Ngoài ra để máy tính có thể chạy thì cần phải nạp vào RAM chương trình, phần này sẽ được nói rõ hơn ở phần sau.

Fetch:

Khi bắt đầu chạy Cp được kích, PC từ giá trị Qn đếm lên Qn+1 (VD: từ ZZZZ đếm lên 0000, từ 0000 đếm lên 0001), Ep được kích, PC xuất giá trị 4-bit ra Bus, giả sử xuất ra giá trị 0001

Lm’ được kích MAR lấy giá trị trên bus, khi này MAR = 0001, bus =ZZZZ, PC = 0002. MAR sẽ chỉ vào địa chỉ 0001 trong RAM.

0001 0D

CE’ được kích, RAM xuất dữ liệu 0D ra bus, khi này giá trị của bus:

Bus = 0000 1101B (0Dh)

Decode:

IR nhận giá trị từ bus khi Li’ (load instructions) được kích, lệnh 8-bit lúc này được chia ra làm 2: 4-bit opcode sẽ đi xuống vào controller để ánh xạ lệnh và biết cần phải làm gì, 4-bit oprand sẽ đi ngược lên MAR để trích xuất và lấy địa chỉ ở trong RAM. Với 0000 opcode được ánh xạ thành mã LDA tức là load, tải dữ liệu vào trong thanh ghi A (phần ánh xạ opcode sẽ được giải thích kỹ hơn tại mục 2.4)

Excute:

Với opcode 0000, controller xuất ra thông tin cho các module để nạp vào thanh ghi A, La’ sẽ được kích hoạt, giá trị trên bus được nạp vào A

2.3 Các module thành phần

2.3.1 Program Counter (Bộ đếm PC)

Nhiệm vụ của bộ đếm chương trình là lưu trữ và gửi địa chỉ bộ nhớ của lệnh tiếp theo cần được lấy và thực thi.

Bộ đếm chương trình, một phần của bộ điều khiển Control Unit, đếm từ 0000 đến 111. Khi chương trình được lưu trữ ở đầu bộ nhớ, với lệnh đầu tiên ở địa chỉ nhị phân 0000, lệnh thứ hai ở địa chỉ 0001, lệnh thứ ba ở địa chỉ 0010,… và cứ chạy liên tục.

Khi máy tính bắt đầu chạy, bộ đếm chương trình sẽ được đặt ở giá trị 0000. Khi quá trình chạy máy tính bắt đầu, bộ đếm chương trình sẽ gửi địa chỉ 0000 đến bộ nhớ, và sau đó tăng thêm 1. Sau khi lệnh đầu tiên được lấy và thực thi, bộ đếm chương trình sẽ gửi địa chỉ tiếp theo, 0001, đến bộ nhớ và tiếp tục tăng thêm. Bằng cách này, bộ đếm chương trình luôn theo dõi lệnh tiếp theo cần được lấy và thực thi.

=> Bộ đếm hay còn được gọi là “con trỏ”, nó nhìn vào danh sách các lệnh và chỉ vào lệnh nào làm trước, lệnh nào làm sau. Trên thực tế, nó trỏ vào địa chỉ trong bộ nhớ memory nơi chứa các dữ liệu (data) hoặc lệnh (instruction)

2.3.2 Input and MAR-Memory Address Register (thanh ghi bộ nhớ địa chỉ)

Ở dưới bộ đếm PC chính là các khối đầu vào (inputs) và thanh ghi bộ nhớ địa chỉ (MAR). Thanh ghi MAR chứa địa chỉ dạng 4-bit của dữ liệu hoặc lệnh, được đặt trong bộ nhớ memory.

Khi hệ thống hoạt động, địa chỉ 4-bit sẽ được lấy từ bộ đếm PC đi qua các bus và sau đó lưu tại MAR. Địa chỉ được lưu này sẽ được gửi tới RAM, sau đó các dữ liệu/lệnh sẽ được đọc

2.3.3 Random-Access Memory (RAM)

SAP-1 sử dụng 16x8 RAM (16 địa chỉ memory chứa tương ứng 8 bits dữ liệu). Thanh RAM có thể được lập trình bằng nhiều cách, trong sách tác giả Paul Malvino

đã dúng các switch công tắc để điều chỉnh các mức cao thấp, hoặc thuận tiện hơn có thể dùng vi điều khiển, tuy nhiên nhóm mô phỏng trên phần mềm nên không dùng các cách trên.

Trong lúc hệ thống hoạt động, thanh ghi RAM sẽ nhận địa chỉ 4-bit từ MAR và đọc các hoạt động đã thực hiện. Bằng cách này lệnh hoặc dữ liệu được đặt trong RAM sẽ được di chuyển ra bus và sẵn sàng cho các bước tiếp theo

2.3.4 Instruction Register (thanh ghi lệnh)

Lấy và chứa lệnh từ RAM thông qua hệ thống bus.

Nội dung trong thanh ghi địa chỉ (IR) được chia thành 2 phần, phần đầu được đưa vào thanh ghi trình tự hoặc thanh điều khiển (Controller/Sequencer), trong khi đó phần còn lại sẽ nằm trong bus chuẩn bị cho bước tiếp theo.

2.3.5 Controller-Sequencer (bộ sắp xếp)

Bộ sắp xếp có mục đích gửi các tín hiệu để kiểm soát SAP-1 nhằm đảm bảo tất cả các module đều hoạt theo đúng mục đích.

Trước khi máy tính hoạt động, tín hiệu $\overline{CLR}$ sẽ được gửi về thanh ghi đếm PC và CLR được gửi về thanh ghi lệnh IR. Điều này sẽ reset PC về giá trị 0000 và xóa tất cả các giá trị trong IR.

Tín hiệu xung CLK được gửi đến tất cả thanh ghi, điều này giúp đồng bộ tất cả các hoạt động trong hệ thống, đảm bảo rằng mọi thứ hoạt động đúng như mong muốn và thiết kế, hay nói cách khác các thanh ghi đều hoạt động theo cạnh lên của xung CLK.

2.3.6 Accumulator (bộ tích lũy)

Bộ tích lũy là một thanh ghi buffer 8-bit để chứa các kết quả trung gian trong quá trình hệ thống máy tính chạy. Bộ tích lũy có 2 ngõ ra. Đầu ra 2-state sẽ đi trực tiếp tới bộ cộng trừ adder-subtractor và đầu ra 3-state sẽ đi vào hệ thống bus

Điều này có nghĩa là word 8-bit accumulator sẽ liên tục được ghi vào bộ cộng trừ, nhưng chỉ xuất hiện trên hệ thống bus khi tín hiệu Ea lên mức cao

2.3.7 ALU - Arithmetic Logic Unit.

Bộ cộng trừ không đồng bộ, sẽ cộng hoặc trừ trực tiếp giá trị từ bộ tích lũy phụ thuộc vào giá trị của tín hiệu Su.

Khi tín hiệu Su ở mức thấp thì sẽ thực hiện phép cộng giữa hai thanh ghi accumulator và thanh ghi B

$$S = A + B$$

Khi tín hiệu ở mức cao.

$$S = A + B'$$

2.3.8 B-Register

Thanh ghi B là một dạng thanh ghi buffer được sử dụng trong các phép tính toán số học, nó hỗ trợ đưa số cộng hoặc trừ từ nội dung của accumulator đến bộ cộng trừ. Khi dữ liệu có sẵn tại bus và tín hiệu Lb có tín hiệu ở mức thấp, tác động xung mức tích cực thì thanh ghi B sẽ nhận và lưu dữ liệu.

2.3.9 Output Resigter

Sau khi thực hiện một phép tính số học, thanh ghi đầu ra nhận và chứa giá trị được lưu trong bộ tích lũy. Kết quả được lưu trong bộ tích lũy sẽ được tải vào thanh ghi đầu ra thông qua hệ thống bus. Cho đến chu kỳ xung tiếp theo thì chu trình hoàn thành, khi đó tín hiệu Ea mức cao và Lo mức thấp. Dữ liệu được tính khi đó có thể được hiện ra bên ngoài.

2.3.10 Binary Display (Màn hình nhị phân)

Màn hình nhị phân là các hàng đèn led, hiển thị kết quả đầu ra bằng cách kết nối với thanh ghi đầu ra.

2.4 Tập lệnh (Instruction set)

Tập lệnh của máy tính là các thao tác cơ bản mà nó có thể thực hiện. Tập lệnh của SAP-1 được mô tả trong bảng dưới đây.

Lệnh	Mô tả
Load	Tải dữ liệu từ bộ nhớ (RAM) vào bộ tích lũy (accumulator)
Add	Cộng dữ liệu từ (RAM) vào bộ tích lũy (accumulator)
Subtract	Trừ dữ liệu từ bộ nhớ (RAM) vào giá trị trong bộ tích lũy
Output	Tải dữ liệu trong bộ tích lũy vào thanh ghi ngõ ra (ouput register)
Halt	Dừng tính toán

Bảng 2.1 Mô tả tập lệnh

LDA, ADD và SUB được gọi là *lệnh liên quan đên bộ nhớ* (memory-reference instructions) bởi vì chúng dùng dữ liệu được lưu trữ trong bộ nhớ. OUT và HLT thì ngược lại, chúng là lệnh không liên quan đến bộ nhớ nên không cần dùng tới dữ liệu trong memory

Khái niệm Mnemonics: Tác giả Malvino giới thiệu đến khái niệm Mnemonics tức là từ gợi nhớ (memory aids) giúp các lập trình viên nhớ về chức năng mà lệnh đó hoạt động. Ví dụ thay vì ghi 0001 để cộng thì có thể ghi thành ADD để cho dễ nhớ.

2.4.1 LDA

LDA viết tắt cho Load the Accumulator, tức "tải vào bộ tích lũy", chức năng của nó đúng như tên gọi, lệnh LDA dùng để tải dữ liệu từ bộ nhớ vào thanh ghi tích lũy, chúng ta có ví dụ: LDA 8H, nghĩa là tải nội dung dữ liệu ở địa chỉ 8H lên accumulator. Nếu nội dung dữ liệu là:

R8 = 1111 0000

Thì sau khi lệnh LDA 8H đã chạy xong kết quả trong thanh ghi accumulator sẽ hiện là:

A = 1111 0000

2.4.2 ADD

ADD cũng là một lệnh trên máy tính SAP-1. Một câu lệnh ADD cộng hoàn chỉnh sẽ gồm cả địa chỉ chứa giá trị cộng vào, ví dụ: ADD 9H tức là cộng giá trị ở địa chỉ 9H của bộ nhớ vào giá trị đang có tại thanh ghi tích lũy accumulator

Cho ví dụ: thực hiện cộng hai số biểu diễn dưới dạng nhị phân, ta có số 2 trong thanh ghi tích lũy và số 3 trong bộ nhớ tại vị trí 9H khi này ta có:

A = 0000 0010

R9 = 0000 011

Đầu tiên giá trị tại R9 sẽ được đưa vào thanh ghi B, giá trị lúc này tại thanh ghi B:

B = 0000 0011

Tiếp theo bộ cộng trừ sẽ thực hiện phép toán cộng giữa A và B

SUM = A+B = 0000 0101

2.4.3 SUB

Tương tự với cấu trúc lệnh cộng ta có lệnh trừ (sub). Cho ví dụ: trừ hai số nhị phân, có số 7 (0111) trong than ghi accumulator và số 3 (0011) trong bộ nhớ memory, địa chỉ là CH. Khi đó:

A = 0000 0111

Rc = 0000 0011

Khi lệnh SUB đươc thực thi, giá trị tại Rc sẽ được tải lên thanh ghi B, khi này giá trị tại B là

B = 0000 0011

Thực hiện phép trừ

DIFF = 0000 0100

Sau khi thực hiện xong phép trừ, giá trị phép tính sẽ được nạp vào accumulator

A = 0000 0100

2.4.4 OUT

Lệnh out sẽ khiến máy tính chuyển xuất dữ liệu từ accumulator ra các cổng xuất dữ liệu, sau khi dữ liệu được xuất ra ngoài, chúng ta có thể thấy dữ liệu đã được xuất.

Lệnh OUT là một lệnh có thể tự hoàn thành chính nó vậy nên lệnh OUT không cần địa chỉ đằng sau bởi vì nó không liên quan đến memory.

2.4.5 HLT

HLT viết tắt cho Halt nghĩa là "dừng lại". Lệnh HLT khiến cho máy tính dừng tính toán dữ liệu. HLT đánh dấu kết thúc chương trình, nó giống như lệnh "$finish" trong Verilog và "return()" trong ngôn ngữ C. Trong máy tính SAP-1 khi kết thúc bất kỳ chương trình nào cũng phải có lệnh HLT nếu không máy tính sẽ bị "trash" (tức là treo), lỗi này xảy ra khi việc tính toán bị lặp liên tục và không dừng được chương trình.

HLT cũng tương tư như OUT, hoàn thành bởi chính nó, nó không liên quan đến memory nên không cần gán địa chỉ trong RAM trong lúc thực thi lệnh.

Ví dụ thực thi chương trình:

Đây là chương trình SAP-1 dưới dạng mnemonics

Địa chỉ	Mnemonics
0H	LDA 9H
1H	ADD AH
2H	ADD BH
3H	SUB CH
4H	OUT
5H	HLT

Bảng 2.2 Bảng ánh xạ Mnemoniac

Dữ liệu tại bộ nhớ memory là

Địa chỉ	Dữ liệu
6H	FFH
7H	FFH
8H	FFH
9H	01H
AH	02H
BH	03H
CH	04H
DH	FFH
EH	FFH
FH	FFH

Bảng 2.3 Địa chỉ và dữ liệu ở RAM

Lệnh đầu tiên sẽ được nạp vào accumulator với nội dung trong địa chỉ 9H, LDA 9H nghĩa là tải nội dung tại địa chỉ 9H lên accumulator, khi này giá trị tại accumulator là:

$$A = 01H$$

Lệnh thứ hai ADD AH cộng vào accumulator giá trị tại địa chỉ AH

$$A = 01H + 02H = 3H$$

Lệnh thứ ba ADD AH cộng vào accumulator giá trị tại địa chỉ BH

$$A = 3H + 3H = 6H$$

Lệnh thứ 4 SUB CH trừ accumlator với giá trị ở địa chỉ CH

$$A = 6H – 04H = 02H$$

Lệnh thứ 5, lệnh OUT được thực thi sẽ xuất giá trị cuối cùng tại thanh ghi accumulator ra các chân output, khi này màn hình sẽ hiển thị:

0000 0010

Lệnh cuối cùng, HLT dừng toàn bộ chương trình tính toán

2.4.2 Mã máy

Để nạp dữ liệu và lệnh vào trong SAP-1 chúng ta cần sử dụng 1 loại ngôn ngữ mà máy tính có thể hiểu. Vì vậy chúng ta phải ánh xã các lệnh Mnemonic sang ngôn ngữ máy, các mã sau khi đã ánh xạ gọi là opcode (operation code)

Mnemonic	Opcode
LDA	0000
ADD	0001
SUB	0010
OUT	1110
HLT	1111

Bảng 2.4 Bảng ánh xạ opcode

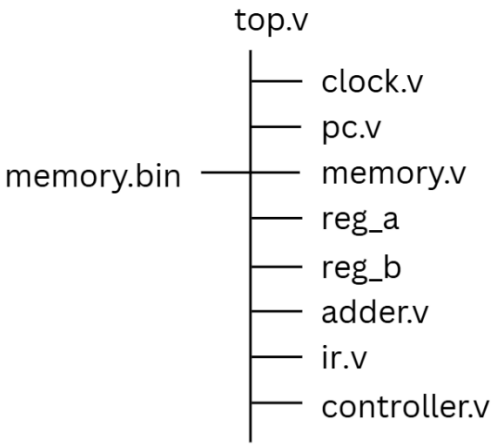
Chúng ta có thể viết chương trình dưới dạng mã hợp ngữ và mã ngôn ngữ máy

Địa chỉ (assembly)	**Nội dung dạng hợp ngữ**	**Địa chỉ (mã máy)**	**Nội dung dạng mã máy**
0H	LDA 9H	0000	0000 1001
1H	ADD AH	0001	0001 1010
2H	ADD BH	0010	0001 1011
3H	SUB CH	0011	0010 1100
4H	OUT	0100	1110 XXXX
5H	HLT	0101	1111 XXXX
6H	FFH	0110	XXXX XXXX
7H	FFH	0111	XXXX XXXX
8H	FFH	1000	XXXX XXXX
9H	01H	1001	0000 0001
AH	02H	1010	0000 0010
BH	03H	1011	0000 0011
CH	04H	1100	0000 0100

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 THIẾT KẾ PHẦN MỀM

Nhóm sẽ thực hiện mô phỏng FPGA qua môi trường Vivado Xilinx, hệ thống sẽ bao gồm 8 module con và 1 module chính để tổng hợp và kết nối các module con lại với nhau



Hình 3.1 sắp xếp cột chương trình

3.1.1 Clock



Hình 3.2 sơ đồ khối Clock

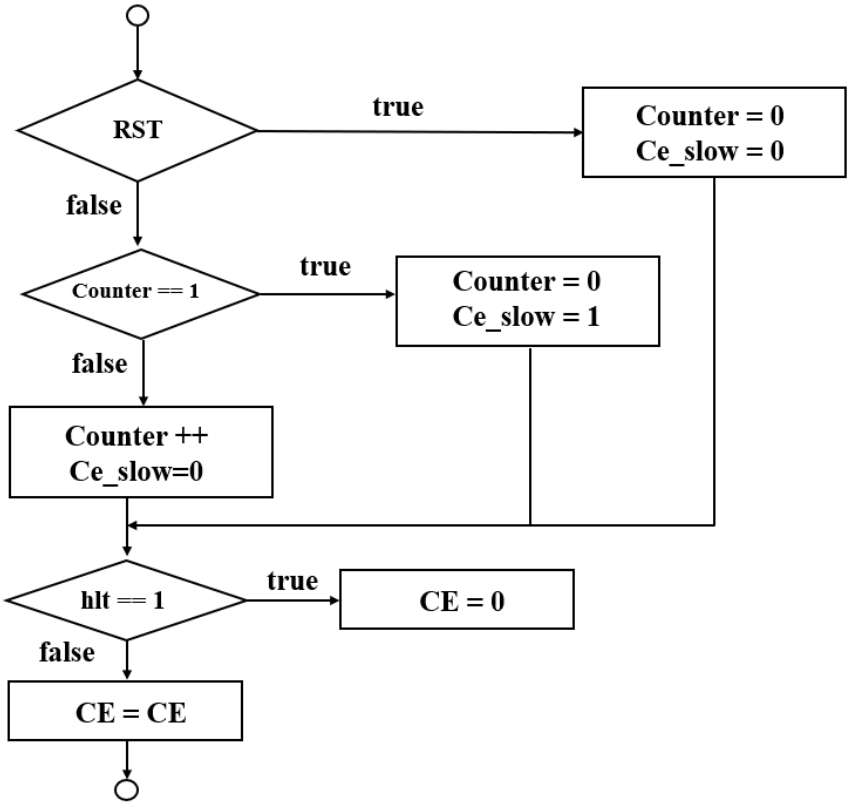
Module này bản chất nó không tạo ra xung mà là điều khiển kiểm soát xung, dựa vào tín hiệu HLT mà có cho phép CLK được chạy không. Module có 2 đầu vào input là clk_in và hlt, 1 đầu ra là clk_out. Khi máy chạy bình thường thì clk_in = clk_out. Khi có tín hiệu dừng (HLT = 1) thì xung ngừng đếm.

Trong môi trường mô phỏng Vivado Simulation thì xung là giả lập do chính đoạn code testbench tạo ra, biến clk_in thành nguồn xung cho toàn bộ hệ thống. Nếu đổ code lên board FPGA thì xung sẽ đến từ linh kiện phần cứng

Khi clk_in (tốc độ 50Mhz) đi vào, nó sẽ được chia làm 2 luồng xung nhịp

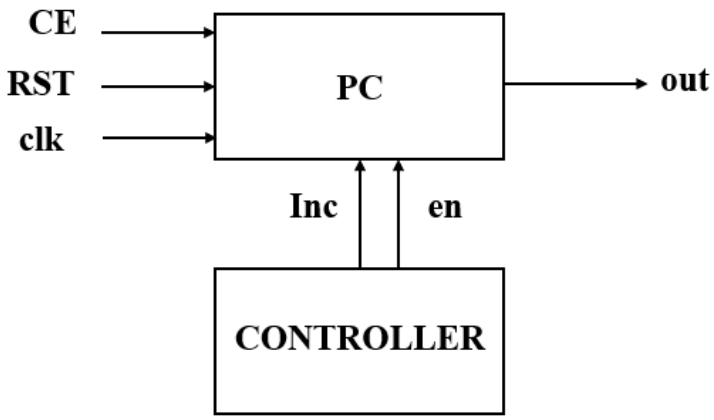
Luồng xung tốc độ cao: clk_in đi thẳng ra clk_out, mục đích để cung cấp xung để duy trì hoạt động cho các chip FPGA

Luồng xung tốc độ thấp: clk_in được đi vào khối always và điều chế thành tín hiệu ce, để tạo ra nhịp để có thể quan sát led nhấp nháy. Tín hiệu ce được điều chế theo kiểu tích lũy, clk_in đi vào bộ đếm counter và bắt đầu đếm, đếm đến nhịp 24.999.999 tức đếm đủ nhịp thì counter reset về 0 và ce_slow lên mức 1



Hình 3.3 lưu đồ giải thuật clock

3.1.2 Program counter



Hình 3.4 Sơ đồ khối Program Counter

Module PC là một bộ đếm 4-bit, nhiệm vụ chính là trỏ đến địa chỉ của lệnh tiếp theo, nó sẽ giữ địa chỉ của lệnh sẽ được thực thi tiếp theo.

PC có 4 chân input bao gồm: clk, reset, CE và inc. Chân clk và CE được kích bởi module clock, inc được kích bởi module controller, chân rst thì trên simulation thì được mô tả bởi testbench, khi đổ trên board thì nó là các nút trên board, khi nhấn nút rst sẽ bị ép về 0.

PC hoạt động như sau: khi xung clock và tín hiệu inc ở mức cao thì giá trị của pc tăng lên 1, còn lại thì giữ nguyên, giá trị trong PC sẽ được đưa ra ngoài để cho các module khác lấy dùng

⇨ Inc = 1 sẽ cho phép PC đếm, EN = 1 sẽ cho phép xuất giá trị ra bus

```
reg[3:0] pc;
always @(posedge clk, posedge rst) begin
    if (rst) begin
            pc <= 4'b0;
    end else if (inc) begin
            pc <= pc + 1;
    end
end
assign out = pc;
```

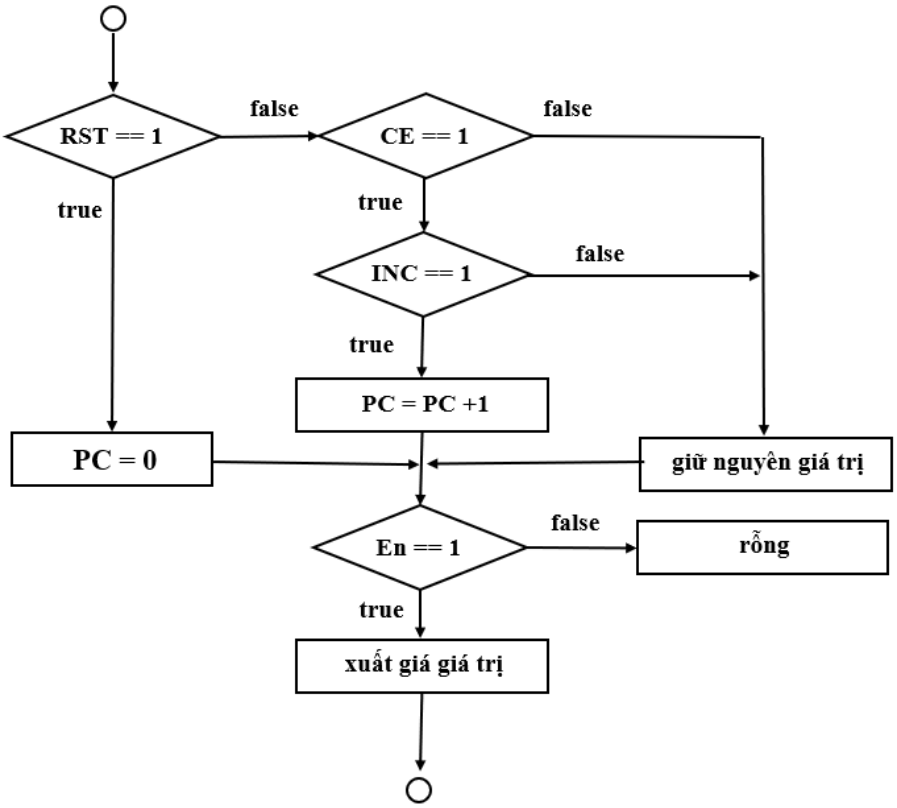
Giải thích:

Reg[3:0] là thanh ghi nội bộ, là một thanh ghi 4 bit, để đánh địa chỉ từ (0000 đến 1111). Khối always là khối logic tuần tự, nó mô tả register/flip-flop sẽ hoạt động như nào, logic bên trong khối always có thể được mô tả như sau

Khi rst = 1, reset pc về 0 (4'b0 tức là 0000)

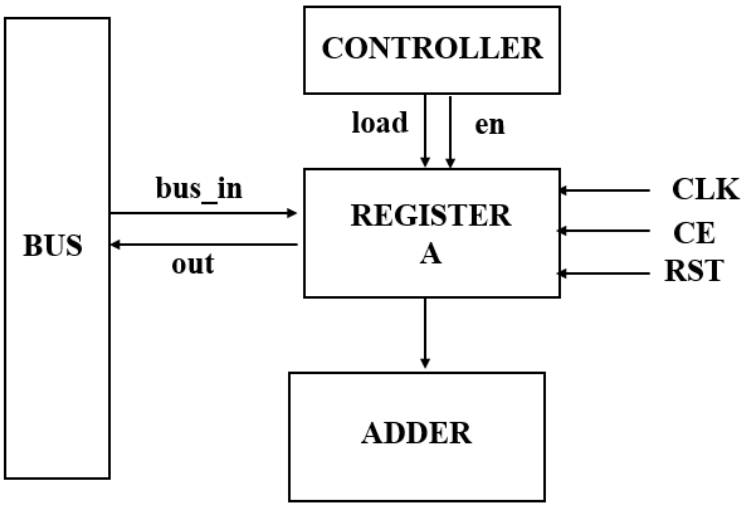
Khi rst = 0 và inc = 1, pc sẽ bắt đầu đếm, giá trị cũ +1

Khi rst = 0 và inc = 0, pc sẽ giữ nguyên giá trị



Hình 3.5 Lưu đồ giải thuật Program Counter

3.1.3 Register A



Hình 3.6 Sơ đồ khối thanh ghi A

Thanh ghi A hay accumulator register, nhiệm vụ của nó là chốt "latch" và lưu trữ 1 giá trị 8-bit được lấy từ w-bus, nó lưu giữ giá trị kết quả trước và sau khi tính toán

Nguồn input là từ ram hoặc adder thông qua bus, điều kiện để vào là khi tín hiệu load lênh mức cao (tín hiệu load điều khiển bởi controller). Output của thanh ghi A có 2 hướng ngõ ra:

Ngõ ra 1: từ A đến Adder, đi trực tiếp tới adder mà không cần thông qua hệ thống bus hay tín hiệu điều khiển khác, bất cứ số nào trong A sẽ ngay lập tức có mặt tại bộ cộng

Ngõ ra 2: từ A đi ra Bus, đi ra hệ thống bus khi có điều kiện khi a_en = 1 (tín hiệu a_en được điều khiển bởi controller)

```
reg[7:0] reg_a;
always @(posedge clk, posedge rst) begin
    if (rst) begin
            reg_a <= 8'b0;
    end else if (load) begin
            reg_a <= bus;
    end
end
```

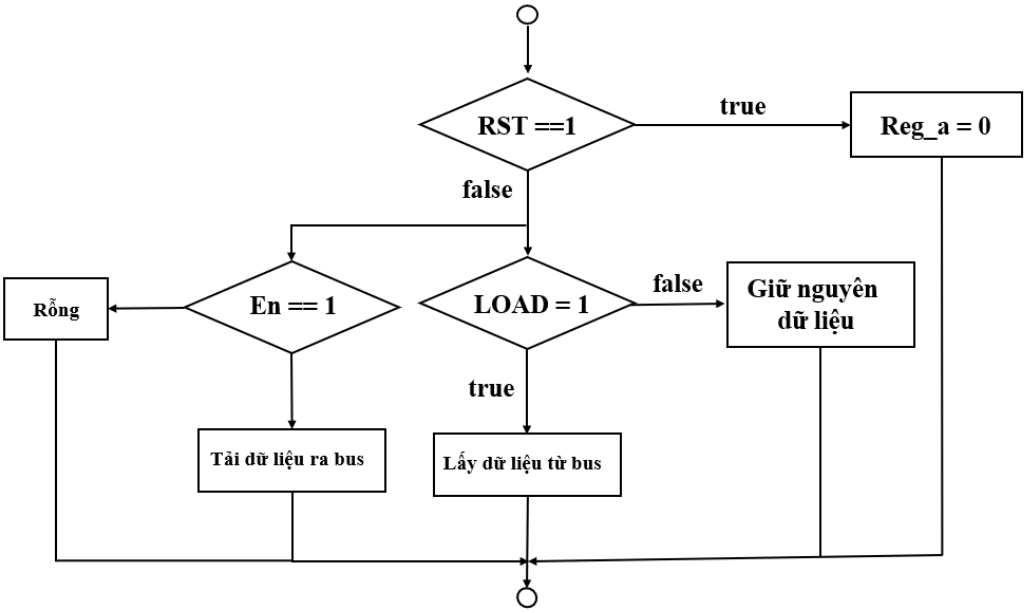
reg[7:0] reg_a; đây là các flip flop dùng để lưu trữ giá trị của thanh ghi

logic bên trong thanh ghi có thể được mô tả như sau:

Khi rst = 1, reg_a reset về 0

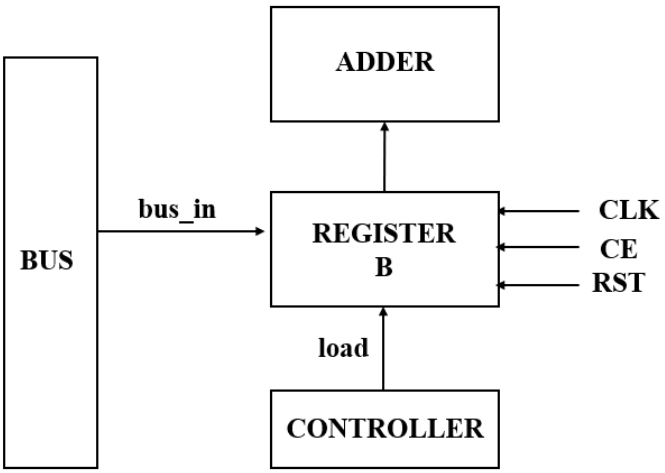
Khi rst = 0 và load = 1, nó sẽ lấy giá trị trên bus và tải vào bên trong giá trị 8-bit

Khi rst = 0 và load = 0 thì giá trị giữ nguyên bất kể giá trị bus thay đổi



Hình 3.7 Lưu đồ của reg_a

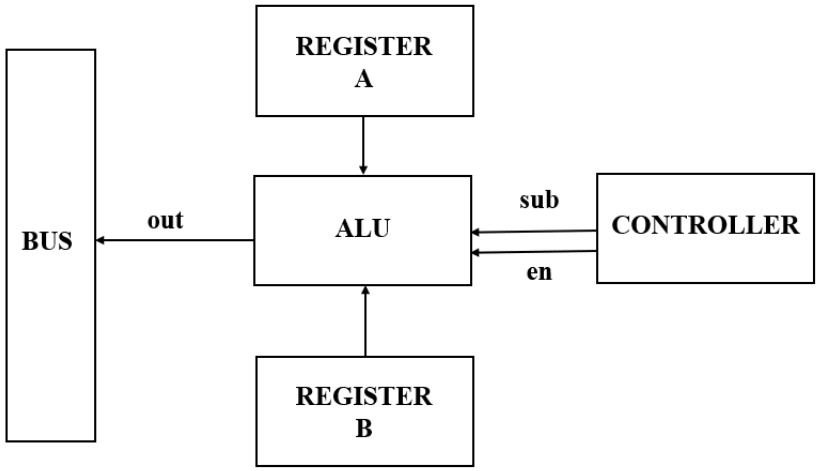
3.1.4 Register B



Hình 3.8 sơ đồ khối thanh ghi B

Về cấu trúc thì thanh ghi gần tương tự với thanh ghi a, thanh ghi b là 1 thanh ghi đệm (buffer) 8-bits, nạp và lưu trữ toán hạng thứ 2 để chuẩn bị cho phép toán

3.1.5 ALU



Hình 3.9 Sơ đồ khối ALU

Module này có chức năng cho phép thực hiện các toán hạng, cụ thể với quy mô SAP-1 là cộng và trừ

Có 3 ngõ vào, kết nối với các module khác như: thanh ghi A, thanh ghi B và Controller, 2 thanh ghi AB sẽ cấp giá trị để tính toán, controller sẽ phát tín hiệu sub để báo cho ALU biết làm phép cộng hay trừ

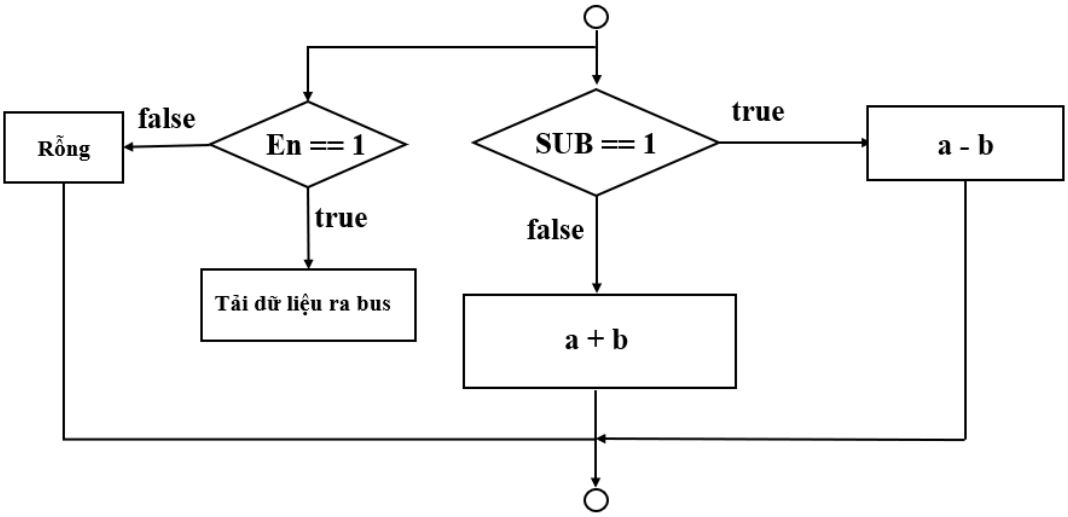
Có 1 ngõ ra Bus, giá trị 8-bit sẽ được xuất ra 1 cách có điều kiện phụ thuộc vào tín hiệu adder_en. Khi adder_en = 1 thì giá trị 8-bit được xuất ra, adder_en = 0 thì đứng im trong ALU

assign out = (sub) ? a-b : a+b;

Có thể hiểu code trên như sau:

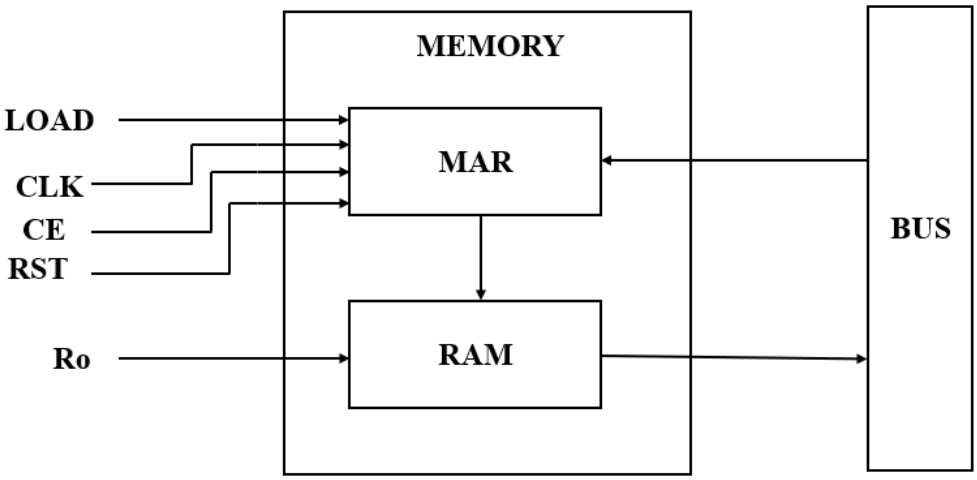
Nếu sub = 1 (sub true) thì out = a – b

Nếu sub = 0 (sub false) thì out = a + b



Hình 3.10 Lưu đồ giải thuật ALU

3.1.6 Bộ nhớ memory



Hình 3.11 Sơ đồ khối Bộ nhớ

Module này mô phỏng bộ nhớ 16x8 (16 ô nhớ, mỗi ô 8 bit) và có thanh ghi 4 bit MAR để trỏ vào bộ nhớ đó. Module này là sự kết hợp giữa hai phần là MAR và RAM

MAR – Memory Address Register - Thanh ghi địa chỉ, đây là thanh ghi 4 bit, nó chỉ cần 4 bit vì bộ nhớ của SAP1 có 16 ô đánh số từ 0 đến 15. Nhiệm vụ chính của MAR là chỉ vào địa chỉ ô nhớ mà CPU muốn truy cập. MAR được nối

với Bus và khi có tín hiệu load (điều khiển bởi controller) thì nó sẽ lấy 4 bit thấp của địa chỉ và lưu lại

Module Memory luông trong trạng thái xuất dữ liệu ra cổng out, MAR chỉ vào đâu thì dữ liệu đi ra ở đó, tuuy nhiên nó không gây nhiễu hệ thống bus là nhờ có controller kiểm soát bằng lệnh mem_en, chỉ khi mem_en = 1 thì dữ liệu từ memory mới đi vào bus chung

reg[3:0] mar; //đây là thanh ghi 4-bits MAR

reg[7:0] ram[0:15]; //đây là bộ nhớ RAM, được khai báo như 1 mảng

$readmemh("program.bin", ram);   // chương trình sẽ tìm file program.bin và nạp vào mảng ram, sau đây là 1 ví dụ mà nhóm đã nạp vào chương trình

0D
1E
2F
E0
F0
00
00
00
00
00
00
00
03
04
02

Bảng 3.1 Chương trình nạp vào trong RAM

Có thể dịch dữ liệu trên như sau:

Địa chỉ	Nội dung	Ý nghĩa lệnh	Diễn giải
00	0D	LDA 13	Nạp giá trị tại ô nhớ số 13 vào Thanh ghi A
01	1E	ADD 14	Cộng giá trị tại ô nhớ số 14 vào Thanh ghi A
02	2F	SUB 15	Trừ giá trị tại ô nhớ số 15 khỏi Thanh ghi A
03	E0	OUT	Xuất giá trị đến thanh ghi ouput
04	F0	HLT	Dừng máy ngay lập tức

Bảng 3.2 Bảng dịch chương trình

Vùng dữ liệu:

Địa chỉ	Giá trị
0D	03
0E	04
0F	02

Bảng 3.3 Bảng vùng dữ liệu trong chương trình

Vậy tổng kết chương trình này là: 3+4-2=5

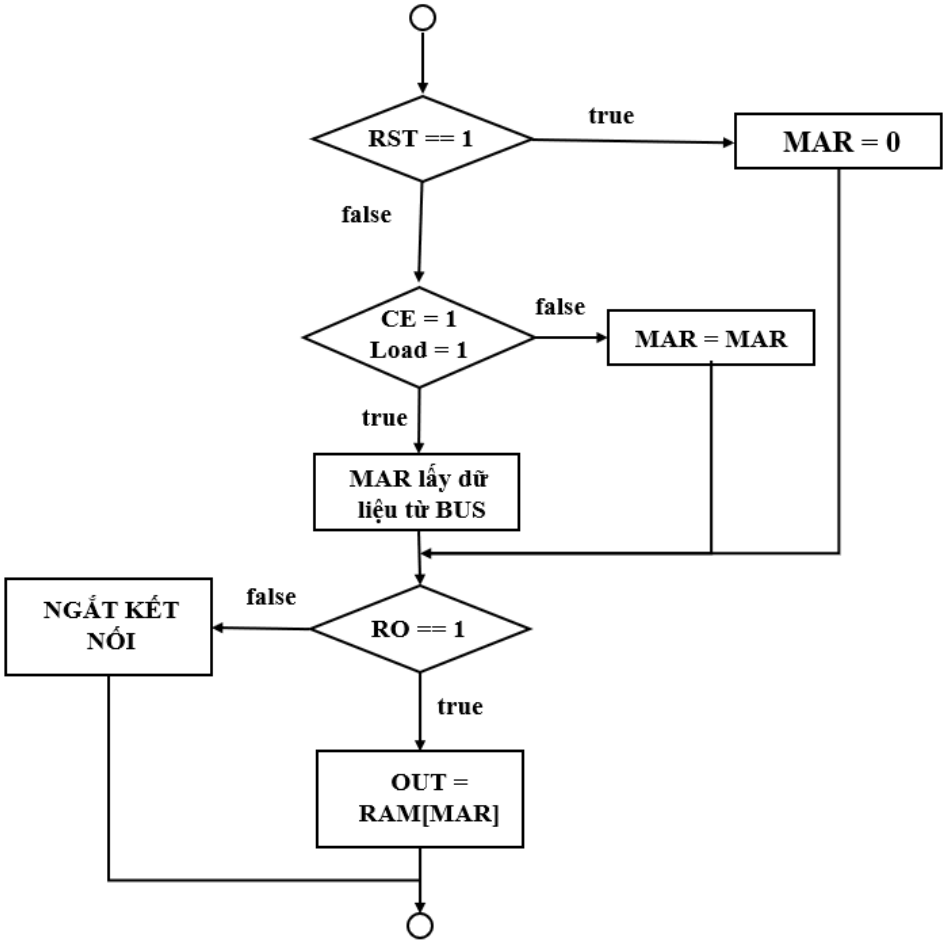
```
always @(posedge clk, posedge rst) begin
    if (rst) begin
            mar <= 4'b0;
    end else if (load) begin
            mar <= bus[3:0];
    end
end
```

//các khối lệnh ở trên mô tả hoạt động của MAR, có thể mô tả như sau:

Nếu reset, mar = 0000

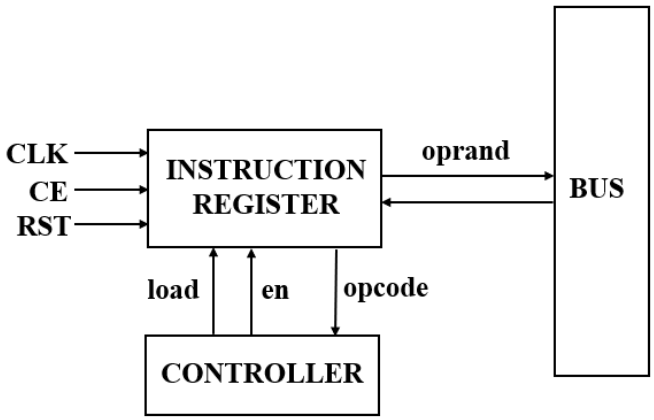
Nếu load = 1 Mar nạp giá trị mới, bit mới được nạp là 4-bits

Nhóm thiết kế RAM ở chế độ chỉ đọc, cpu không thể ghi đè lên RAM



Hình giải thuật Memory

3.1.7 IR-thanh ghi lệnh



Hình 3.12 Sơ đồ khối thanh ghi lệnh

Ir là thanh ghi 8 bits, nó nạp mã lệnh từ bus và giữ lệnh này trong khi controller thực thi lệnh. Ngõ ra 8-bits của nó được đưa tới bộ điều khiển:

4 bit opcode sẽ cho bộ điều khiển biết cần phải làm gì, sẽ được gửi trực tiếp đến controller để giải mã

4 bit operand sẽ cho bộ điều khiển biết dữ liệu liên quan đến lệnh, được đưa lại bus để nạp vào MAR

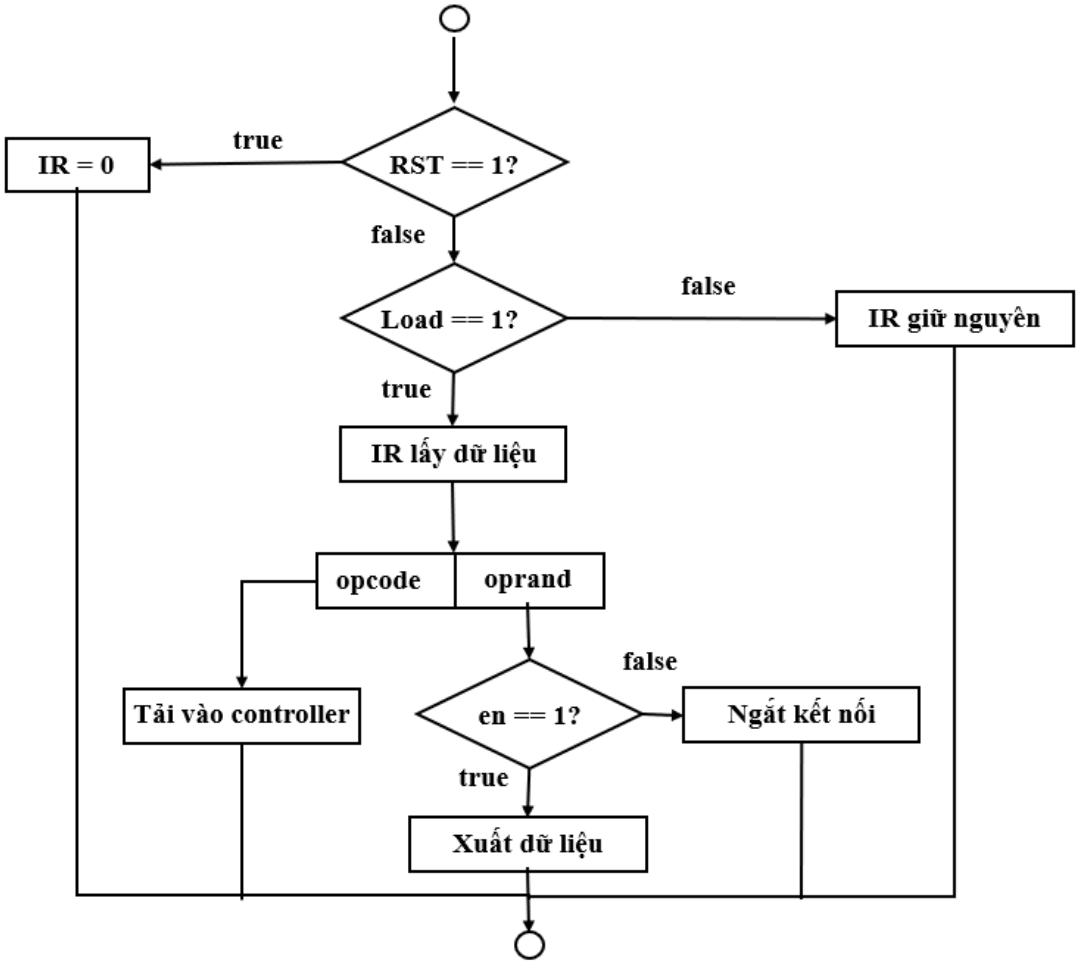
```
always @(posedge clk, posedge rst) begin
    if (rst) begin
            ir <= 8'b0;
    end else if (load) begin
            ir <= bus;
    end
end
```

có thể hiểu lệnh trên như sau:

Nếu rst =1, ir bị reset về 0

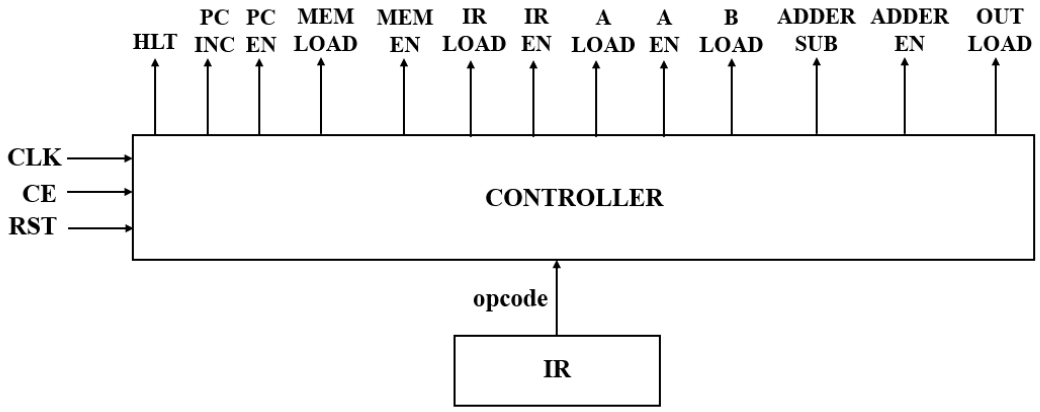
Nếu rst = 0 và load = 1, ir nạp mã lệnh 8-bits hiện có trên bus

Nếu rst = 0 và load = 0 sẽ giữ nguyên lệnh đang lưu



Hình 3.13. Giải thuật module IR

3.1.8 Bộ điều khiển



Hình 3.14 Sơ đồ khối Bộ điều khiển

Module controller là trung tâm của SAP-1, nó phát tín hiệu điều khiển, ra lệnh cho các module khác biết cần phải làm gì trong khoảng thời gian nào. Module này hoạt động bằng cách đếm qua 6 giai đoạn (từ stage 0 đến stage 5) cho mỗi chu kỳ lệnh. Tại mỗi giai đoạn, nó sẽ xem xét lệnh hiện tại (opcode) là gì và quyết định xem tín hiệu điều khiển nào cần được bật lên.

Localparam // đặt tên cho các bit điều khiển

Tên bit điều khiển	Ý nghĩa
SIG_HLT	Dừng mọi hoạt động
SIG_PC_INC	Tăng PC
SIG_PC_EN	PC xuất data ra bus
SIG_MEM_LOAD	Tải vào MAR
SIG_MEM_EN	Cho phép RAM xuất ra bus
SIG_IR_LOAD	Tải vào IR
SIG_IR_EN	IR xuất 4bits ra bus
SIG_A_LOAD	Tải vào thanh ghi A
SIG_A_EN	Tải giá trị ở thanh ghi A vào bus
SIG_B_LOAD	Tải giá trị ở thanh ghi B vào bus
SIG_ADDER_SUB	Cho phép ALU thực hiện phép trừ
SIG_ADDER_EN	Cho phép ALU xuất kết quả ra bus
SIG_OUT_REG	Cho phép thanh ghi output nhận giá trị

Bảng 3.4 Bảng chức năng của local param

```
if (rst) begin
        stage <= 0;
    end else begin
        if (stage == 5) begin
            stage <= 0;
        end else begin
            stage <= stage + 1;
        end
    end
```

end

Reg[2:0] stage; dùng để lưu trữ lại stage hiện tại đang được thực thi, có thể xem đoạn code trên là một bộ đếm, nếu stage = 5 thì sẽ reset về 0.Các stage bao gồm: Fetch (stage 0,1,2); Decode (stage3); Excute (stage 4,5). Các bước stage được mô tả như sau:

Stage 0:

SIG_PC_EN = 1 // PC ra bus

SIG_MEM_LOAD = 1 // nạp vào MAR

=> chuyển địa chỉ của lệnh từ PC sang MAR

Stage 1:

SIG_PC_INC = 1 //tăng PC

=> sẵn sàng cho lệnh tiếp theo

Stage 2:

SIG_MEM_EN = 1 //PC ra bus

SIG_IR_LOAD = 1 //nạp vào IR

=> Nạp mã lệnh 8-bits từ RAM vào thanh ghi lệnh

Stage 3:

SIG_IR_EN = 1; (IR[3:0] ra Bus)

SIG_MEM_LOAD = 1; (Nạp vào MAR)

=>MAR <= IR[3:0]. Lấy 4 bit địa chỉ từ lệnh và nạp vào MAR để trỏ đến dữ liệu

Stage 4:

Nếu là lệnh LDA:

SIG_MEM_EN = 1; (RAM ra Bus)

SIG_A_LOAD = 1; (Nạp vào A)

=> Tải dữ liệu từ RAM vào Thanh ghi A. (Lệnh LDA kết thúc).

Nếu là lệnh ADD hoặc SUB

SIG_MEM_EN = 1; (RAM ra Bus)

SIG_B_LOAD = 1; (Nạp vào B)

=>Tải dữ liệu từ RAM vào Thanh ghi B (để chuẩn bị làm phép toán).

Stage 5:

Nếu là ADD:

SIG_ADDER_EN = 1; (ALU ra Bus)

SIG_A_LOAD = 1; (Nạp vào A)

=> Lấy kết quả A+B từ ALU và nạp ngược vào Thanh ghi A.

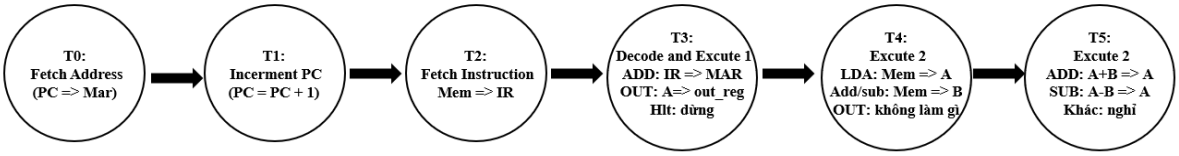
Nếu là SUB:

SIG_ADDER_SUB = 1; (Ra lệnh Trừ)

SIG_ADDER_EN = 1; (ALU ra Bus)

SIG_A_LOAD = 1; (Nạp vào A)

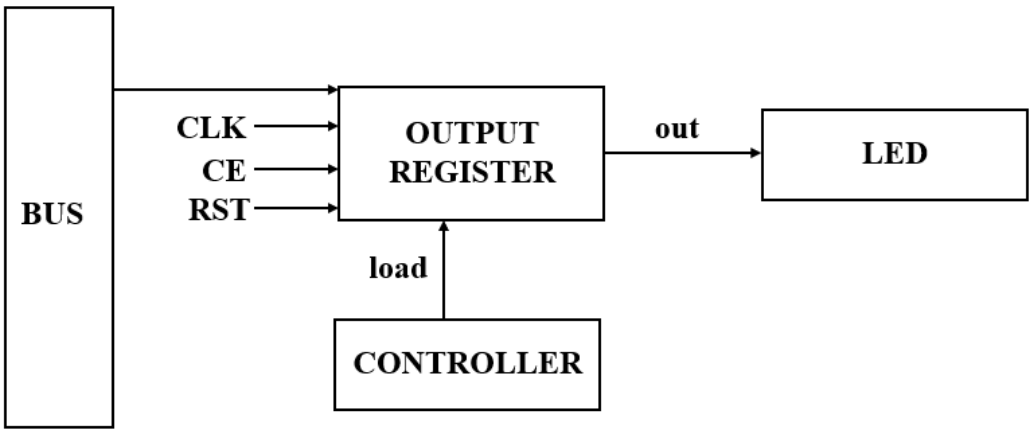
=> Lấy kết quả A-B từ ALU và nạp ngược vào Thanh ghi A.



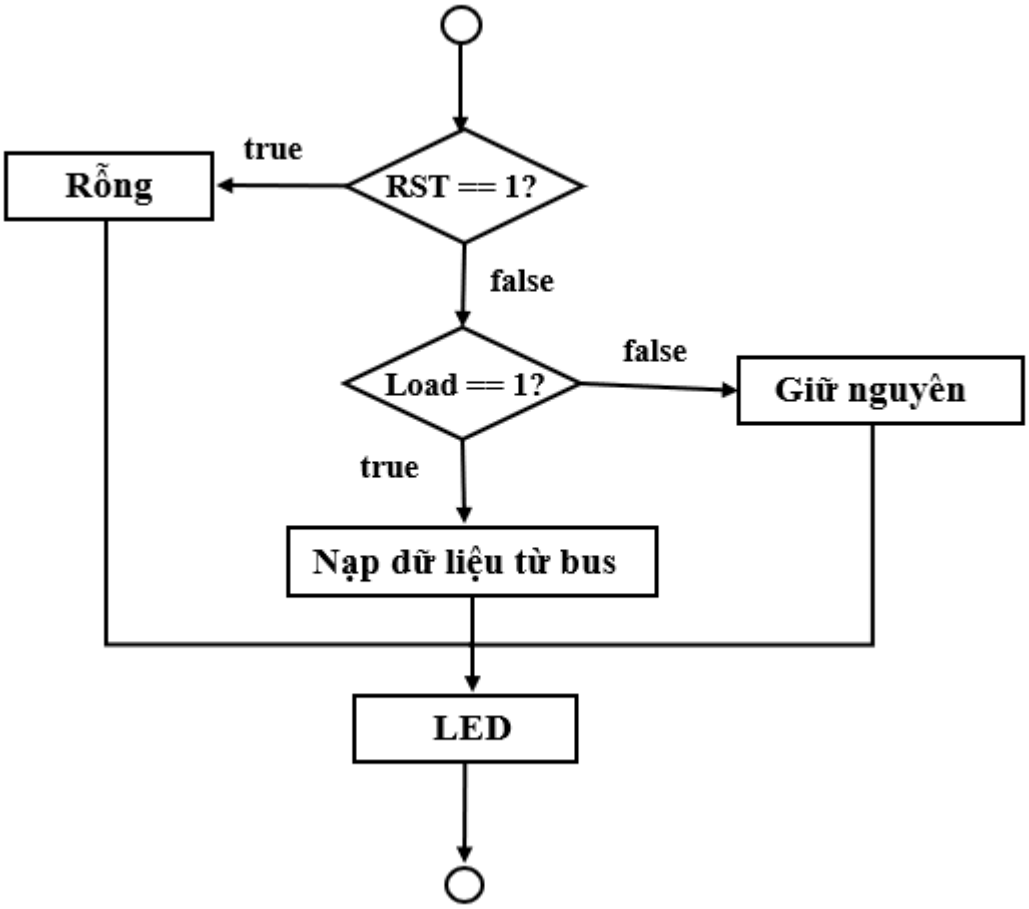
Hình 3.14 Hình chuỗi đếm state trong IR

3.1.9 THANH GHI ĐẦU RA (OUTPUT REGISTER)

Đây là một cổng giao tiếp duy nhất giữa máy tính bên trong và người dùng, khi controller ra lệnh load, OR sẽ ghi lại con số đang có trên bus và truyền xuống các led



Hình 3.15 Sơ đồ thanh ghi đầu ra



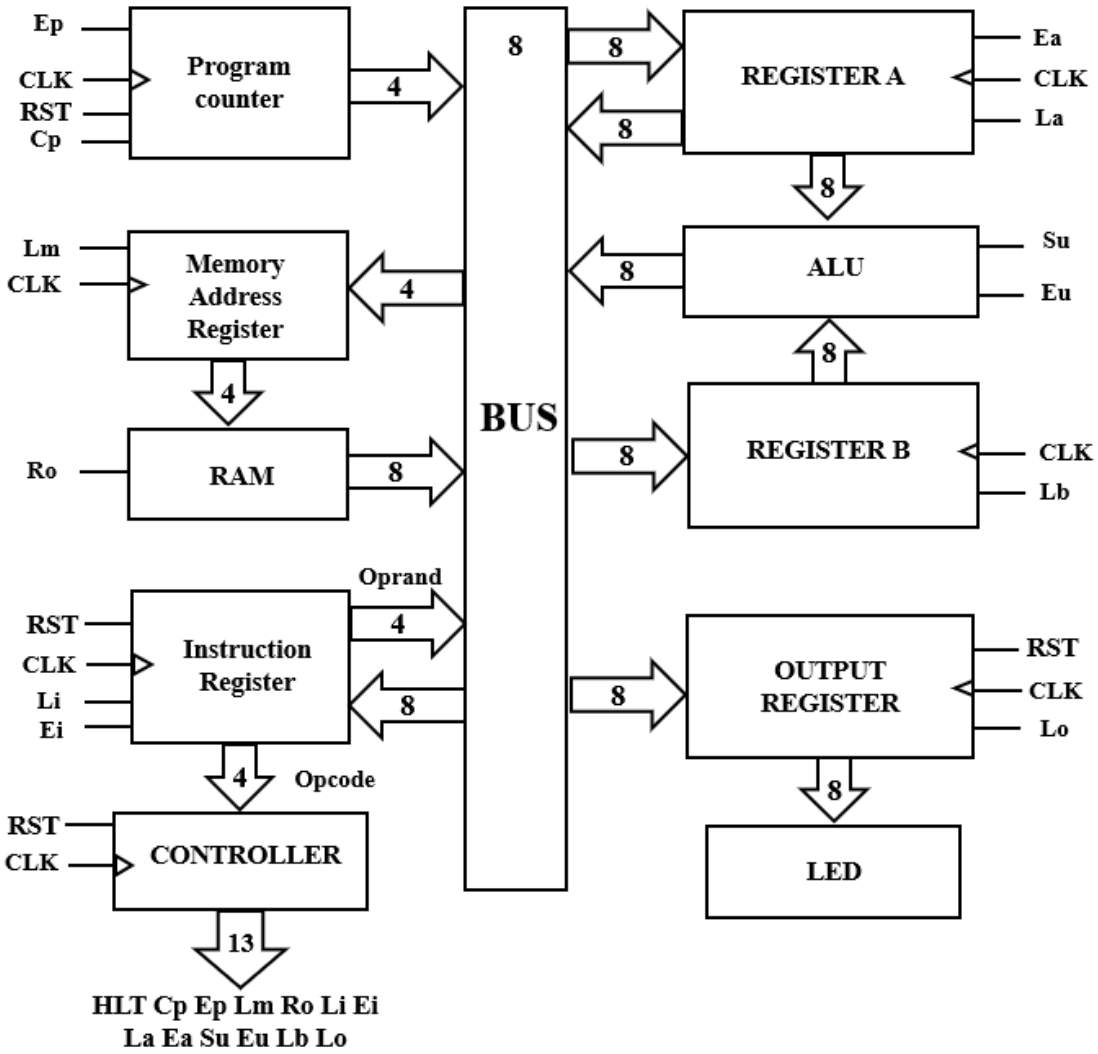
3.1.10 Kết nối tất cả các module

Cuối cùng để kết nối tất cả các module lại với nhau thông qua module top, đây là module quan trọng nhất, có vai trò như một main board để kết nối với các module còn lại

```
always @(*) begin
    if (ir_en) begin
            bus = ir_out;
    end else if (adder_en) begin
            bus = adder_out;
    end else if (a_en) begin
            bus = a_out;
    end else if (mem_en) begin
            bus = mem_out;
    end else if (pc_en) begin
```

```
            bus = pc_out;
        end else begin
            bus = 8'b0;
        end
    end
```

Khối mã lệnh trên là khối tổ hợp MUX đảm bảo rằng tại 1 thời điểm chỉ có duy nhất một module được phép xuất dữ liệu ra bus



Hình 3.16 Tổng quan mô hình SAP-1

3.2 BẢNG TRẠNG THÁI TỪNG MODULE

3.2.1 CLOCK

Clk_in	hlt	Clk_out	Trạng thái
0	0	0	Đếm
1	0	1	Đếm
0	1	0	Ngưng
1	1	0	ngưng

Bảng 3.5 Bảng trạng thái module Clock

3.2.2 PC

rst	ce	inc	en	Out (hiện tại)	Out (kế tiếp)	Trạng thái
1	x	x	x	x	0000	Xóa về 0, ngắt bus
0	0	x	x	Q	Q	Dừng clock, giữ nguyên
0	1	0	0	Q	Q	Dừng đếm,ngắt bus
0	1	0	1	Q	Q	Dừng đếm, xuất giá trị
0	1	1	0	Q	Q+1	Đếm, không xuất
0	1	1	1	Q	Q +1	Đếm và xuất ra bus
0	1	1	1	0000	0001	Đếm lên
0	1	1	1	0001	0010	Đếm lên
0	1	1	1	0010	0011	Đếm lên
0	1	1	1	0011	0100	Đếm lên
0	1	1	1	0100	0101	Đếm lên
0	1	1	1	0101	0110	Đếm lên
0	1	1	1	0110	0111	Đếm lên
0	1	1	1	0111	1000	Đếm lên
0	1	1	1	1000	1001	Đếm lên
0	1	1	1	1001	1010	Đếm lên
0	1	1	1	1010	1011	Đếm lên

0	1	1	1	1011	1100	Đếm lên
0	1	1	1	1100	1101	Đếm lên
0	1	1	1	1101	1110	Đếm lên
0	1	1	1	1110	1111	Đếm lên
0	1	1	0	1111	0000	Đếm ngược về 0

Bảng 3.6 Trạng thái Module PC

3.2.3 ALU

Eu	Sub	Kết quả	TRẠNG THÁI
0	X	A	Nghỉ
1	0	A+B	Cộng
1	1	A-B	Trừ

Bảng 3.7 Bảng trạng thái ALU

Eu	Sub	A	B	ALU	Out	Trạng thái
0	x	A	B			Nghỉ
1	0	3	4	3+4	7	Tổng
1	1	7	2	7-2	5	Hiệu
1	1	5	10	5-10	FB 1111 1011	-5 dưới dạng Bù 2 là số 251 hoặc FB trong hệ hex
1	0	200	100	200+100	2c(44) 1 0010 1100	Tràn 1 bit đầu tiên, kết quả chỉ còn: 0010 1100

Bảng 3.8 Ví dụ phép tính trong ALU

3.2.4 IR

Lệnh LDA 0D

rst	ce	load	ei	Bus_in(hex)	Bus_in(bin)	Opcode	Operand
0	1	0	1	0D	0000 1101	0000	1101

Lệnh ADD 1E

rst	ce	load	ei	Bus_in(hex)	Bus_in(bin)	Opcode	Operand

0	1	0	1	1E	0001 1110	0001	1110

Lệnh SUB 2F

rst	ce	load	ei	Bus_in(hex)	Bus_in(bin)	Opcode	Operand
0	1	0	1	2F	0010 1111	0010	1111

Lệnh HLT F0

rst	ce	load	ei	Bus_in(hex)	Bus_in(bin)	Opcode	Operand
0	1	0	1	F0	1111 0000	1111	0000

3.2.5 CONTROLLER

12	11	10	9	8	7	6	5	4	3	2	1	0
Load out	halt	PC INC	PC EN	MAR Load	RAM out	IR load	IR EN	A Load	A En	B Load	sub	ALU en
Lo	hlt	Cp	Ep	Lm	CE	Li	Ei	La	Ea	Lb	Su	Eu

Bảng 3.9 Chú thích các chân tín hiệu

Chu kỳ đón lệnh fetch

state	12	11	10	9	8	7	6	5	4	3	2	1	0	TT
0	0	0	0	1	1	0	0	0	0	0	0	0	0	PC =>MAR
1	0	0	1	0	0	0	0	0	0	0	0	0	0	PC + 1
2	0	0	0	0	0	1	1	0	0	0	0	0	0	RAM => IR

Bảng 3.9a chu kỳ state lệnh fetch

Lệnh LDA

state	12	11	10	9	8	7	6	5	4	3	2	1	0	TT
3	0	0	0	0	1	0	0	1	0	0	0	0	0	IR =>MAR
4	0	0	0	0	0	1	0	0	1	0	0	0	0	RAM => A
5	0	0	0	0	0	0	0	0	0	0	0	0	0	Nghỉ

Bảng 3.9b chu kỳ state lệnh load

Lệnh ADD

state	12	11	10	9	8	7	6	5	4	3	2	1	0	TT
3	0	0	0	0	1	0	0	1	0	0	0	0	0	IR =>MAR
4	0	0	0	0	0	1	0	0	0	0	1	0	0	RAM=>B
5	0	0	0	0	0	0	0	0	1	0	0	0	1	ALU=>A

Bảng 3.9c chu kỳ state lệnh ADD

Lệnh SUB

state	12	11	10	9	8	7	6	5	4	3	2	1	0	TT
3	0	0	0	0	1	0	0	1	0	0	0	0	0	IR =>MAR
4	0	0	0	0	0	1	0	0	0	0	1	0	0	RAM=>B
5	0	0	0	0	0	0	0	0	1	0	0	1	1	ALU=>A

Bảng 3.9d chu kỳ state lệnh SUB

Lệnh OUT

state	12	11	10	9	8	7	6	5	4	3	2	1	0	TT
3	1	0	0	0	0	0	0	0	0	1	0	0	0	A=>out_reg

Bảng 3.9e state lệnh OUT

Lệnh HLT

state	12	11	10	9	8	7	6	5	4	3	2	1	0	TT
3	0	1	0	0	0	0	0	0	0	1	0	0	0	Dừng

Bảng 3.9f state lệnh HALT

3.2.6. MEMORY

rst	ce	load	ro	Bus (in)	MAR hiện tại	MAR (kế tiếp)	Out (Data)	Trạng thái

1	X	X	X	X	X	0000	Z	Reset: Xoá MAR về 0, ngắt bus
0	0	X	0	X	Q	Q	Z	Dừng clock, ngắt bus
0	0	X	1	X	Q	Q	RAM[Q]	Dừng clock nhưng vẫn xuất tín hiệu
0	1	0	0	X	Q	Q	Z	Nghỉ, ngắt bus
0	1	0	1	X	Q	Q	RAM[Q]	Đọc dữ liệu ra bus
0	1	1	0	A	Q	A	Z	Nạp dữ liệu A vào MAR
0	1	1	1	X	Q	X	X	Cấm: xung đột bus (vừa nạp, vừa xuất)

Bảng 3.10 Bảng trạng thái tổng quát Memory

Bảng trạng thái theo lệnh:

LDA

rst	ce	load	ro	Bus (in)	MAR hiện tại	MAR kế tiếp	Out (Data)	Trạng thái
0	1	1	0	00	XX	00	Z	Nạp MAR = 00
0	1	0	1	x	00	00	0D	Đọc lệnh LDA
0	1	1	0	0D	00	0D	Z	Nạp MAR = 13 (OD)
0	1	0	1	X	0D	0D	03	Đọc Data (lấy số 03)

Bảng 3.10a Bảng trạng thái lệnh LDA

ADD:

rst	ce	load	ro	Bus (in)	MAR (hiện tại)	MAR kế tiếp	Out (Data)	Trạng thái
0	1	1	0	01	0D	01	Z	Nạp MAR = 01
0	1	0	1	x	01	01	1E	Đọc lệnh ADD
0	1	1	0	0E	01	0E	Z	Nạp MAR = 14 (OE)
0	1	0	1	X	0E	0E	04	Đọc Data (lấy số 04)

Bảng 3.10b Bảng trạng thái lệnh ADD

SUB:

rst	ce	load	ro	Bus (in)	MAR (hiện tại)	MAR (kế tiếp)	Out (Data)	Trạng thái
0	1	1	0	02	0E	02	Z	Nạp MAR = 02
0	1	0	1	x	02	02	2F	Đọc lệnh SUB
0	1	1	0	0F	02	0F	Z	Nạp MAR = 15 (OF)
0	1	0	1	X	0E	0E	04	Đọc Data (lấy số 04)

Bảng 3.10c Bảng trạng thái lệnh SUB

OUT:

rst	ce	load	ro	Bus (in)	MAR (hiện tại)	MAR (kế tiếp)	Out (Data)	Trạng thái
0	1	1	0	03	0F	03	Z	Nạp MAR = 03
0	1	0	1	x	03	03	E0	Đọc lệnh OUT

Bảng 3.10c Bảng trạng thái lệnh OUT

HLT:

rst	ce	load	ro	Bus (in)	MAR (hiện tại)	MAR (kế tiếp)	Out (Data)	Trạng thái
0	1	1	0	04	03	04	Z	Nạp MAR = 03
0	1	0	1	x	04	04	F0	Đọc lệnh HLT

Bảng 3.10d Bảng trạng thái lệnh HLT

Q: giá trị hiện tại đang nằm trong thanh ghi MAR

A: giá trị bất kỳ đang có trên bus (address)

Z: trạng thái trở kháng cao

RAM[Q]: dữ liệu nằm trong ô nhớ có địa chỉ là Q

3.2.7 THANH GHI A

rst	ce	load	Bus_in	Giá trị hiện tại	Giá trị tiếp	Trạng thái
1	x	X	xxxx xxxx	xxxx xxxx	0000 0000	reset
0	0	X	Xxxx xxxx	Giá trị cũ	Giá trị cũ	Clock dừng
0	1	0	Xxxx xxxx	Giá trị cũ	Giá trị cũ	hold
0	1	1	Giá trị hiện tại	Giá trị cũ	Giá trị hiện tại	Load vào A

Bảng 3.11 Bảng trạng thái thanh ghi A

3.2.8 THANH GHI OUTPUT

rst	ce	load	Bus_in	Giá trị hiện tại	Giá trị tiếp	Trạng thái
1	x	x	xxxx xxxx	xxxx xxxx	0000 0000	reset
0	0	x	xxxx xxxx	Giá trị cũ	Giá trị cũ	Clock dừng
0	1	0	xxxx xxxx	Giá trị cũ	Giá trị cũ	hold
0	1	1	Giá trị hiện tại	Giá trị cũ	Giá trị hiện tại	Load vào A

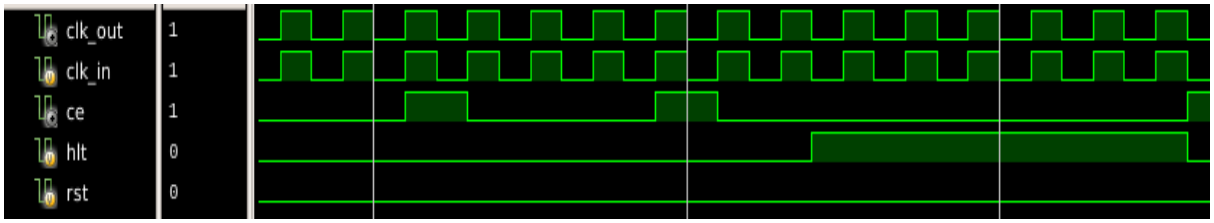
Bảng 3.12 Bảng trạng thái thanh ghi output

CHƯƠNG 4: KẾT QUẢ

4.1 Kết quả chạy thử nghiệm của các module

Nhóm sẽ thực hiện kiểm thử độ chính xác của từng module đơn lẻ, nhóm sẽ dựa vào bảng trạng thái đã đưa ra ở mục 3.2 để kiểm tra

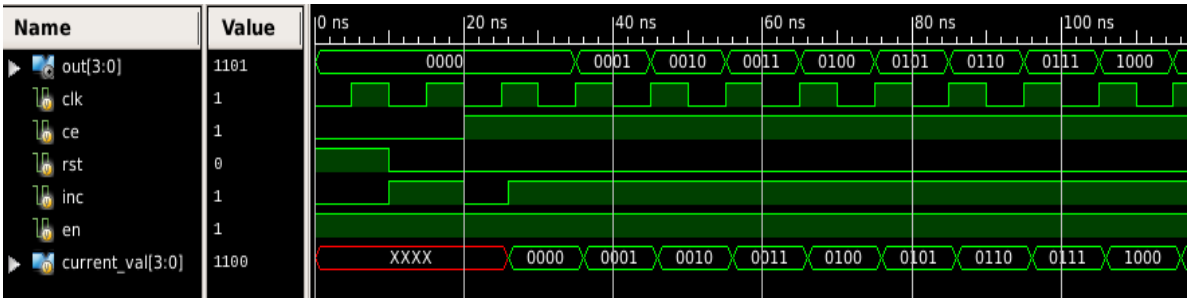
4.1.1 CLOCK TEST



Hình 4.1 Waveform module clock

Ở module clock, nhóm chia thành 2 hướng xung, xung clk_out tốc độ cao dùng để nuôi các linh kiện trên kit, và 1 hướng xung ce (clock enable) dùng để quan sát logic. Qua kiểm tra có thể thấy khi hlt = 1 thì ce ngưng đếm.

4.1.2 PROGRAM COUNTER TEST



Hình 4.2 Waveform module PC

Current _value là giá trị hiện tại trong PC

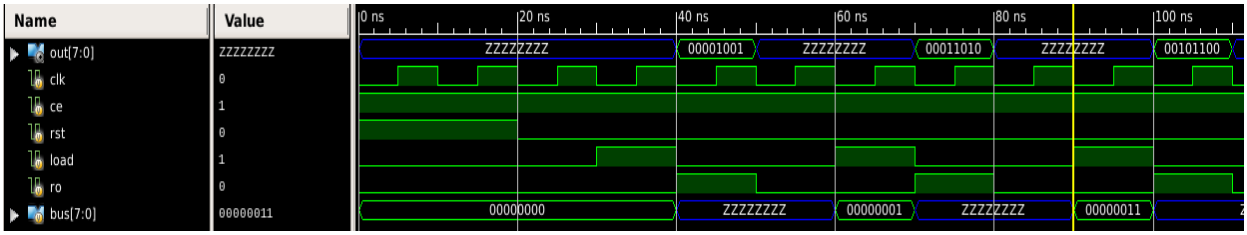
Out [3:0] là giá trị trên bus sau khi đã được xuất ra từ PC

Inc: cho phép PC đếm

En: cho phép PC xuất giá trị ra bus

Khi inc = 1 thì pc bắt đầu đếm, khi ce = 1 thì module cho phép xuất giá trị ra bus

4.1.3 MEMORY TEST



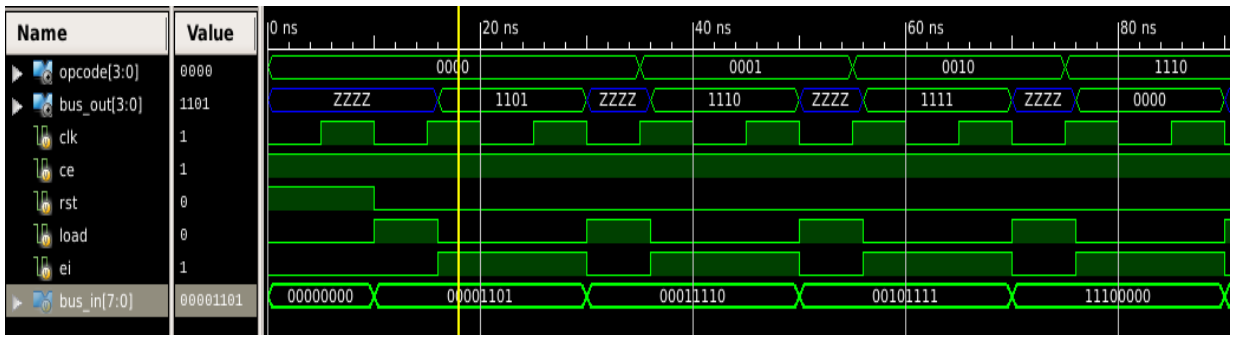
Hình 4.3 Waveform module Memory

Ở module Memory, mục tiêu là kiểm tra khả năng xuất dữ liệu từ RAM ra bus. Với lệnh load khi được kích sẽ cho phép lấy địa chỉ từ bus vào, khi Ro được kích thì sẽ xuất dữ liệu/lệnh từ trong RAM ra ngoài bus,nhóm sẽ giả lập nạp vào một đoạn chương trình để kiểm tra

Địa chỉ (assembly)	Nội dung dạng hợp ngữ	Địa chỉ (mã máy)	Nội dung dạng mã máy
0H	LDA 9H	0000	0000 1001
1H	ADD AH	0001	0001 1010
2H	ADD BH	0010	0001 1011
3H	SUB CH	0011	0010 1100
4H	OUT	0100	1110 XXXX
5H	HLT	0101	1111 XXXX

Hình 4.4 Chương trình kiểm thử Memory

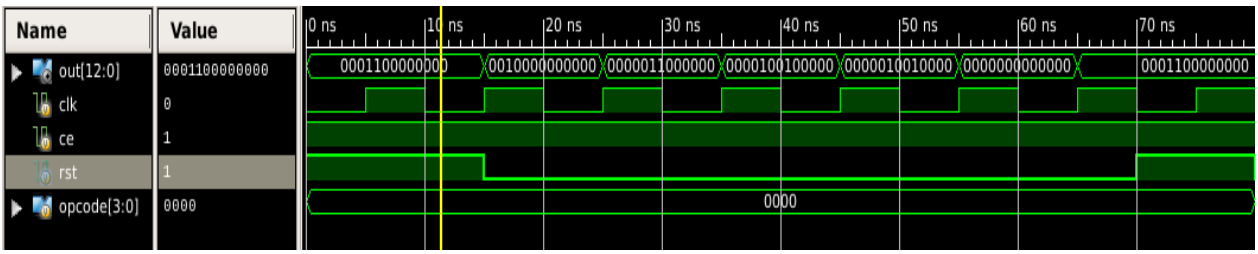
4.1.4 IR TEST



Hình 4.5 Waveform module IR

Có thể tóm tắt quá trình hoạt động của module IR như sau: IR nhận lệnh 8-bits từ bus_in khi tín hiệu load = 1, sau đó thì IR sẽ bắt đầu phân tách thành 4-bits opcode để truyền xuống Controller, 4-bits operand đi ra bus khi tín hiệu Ei = 1. VD: khi trên bus có lệnh 0Dh hay 0000 1101b, khi Ei = 1 thì bus[3:0] (tức operand) = 1, khi load = 1 thì opcode = 0000

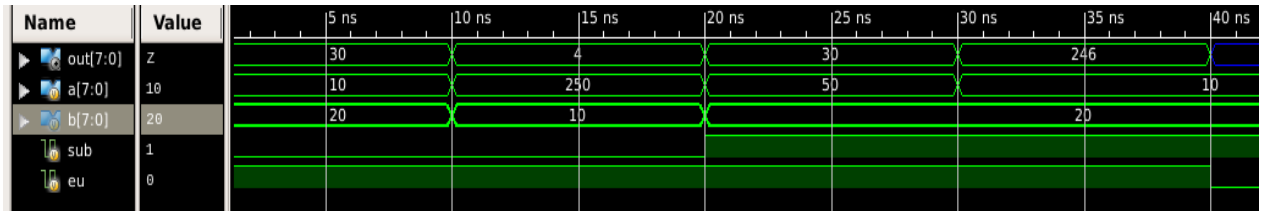
4.1.5 CONTROLLER TEST



Hình 4.6 Waveform module Controller

Để kiểm tra module Controller bằng cách giả lập chân opcode để đưa mã vào sau đó kiểm tra 12-bits đầu ra theo từng state từ state 1 đến state 5

4.1.6 ALU TEST



Hình 4.7 Waveform module ALU

Để kiểm tra moduler adder bằng cách giả lập 2 chân đầu vào a,b gán cho mỗi chân giá trị để tính, module ALU mặc định là luôn thực hiện phép cộng, khi tín hiệu sub = 1 thì ALU mới thực hiện phép trừ

Ta cho ví dụ như trên hình, đầu tiên với ví dụ cộng trừ thường

$$A + B = 10 + 20 = 30$$

$$A – B = 50 – 20 = 30$$

Đây là trường hợp cộng tràn giới hạn của ALU trong mô hình này là 8-bits vì vậy khi mà phép toán vượt ngưỡng 8-bit thì bit bị tràn sẽ bị mất, 0001 0000 0100 sẽ bị rơi mất 1 bit nên còn 0000 0100

$$A + B = 10 + 250 = 4$$

Ngoài ra còn trường hợp trừ số âm, vì máy tính không thể hiểu các giá trị âm nên sẽ thực hiện theo phương thức trừ bù 2

$$10 – 20 = 0000\ 1010 – 0001\ 0100$$

Để thực hiện bù 2, ta sẽ cần bù 1 cho -20

$$0001\ 0100 \Rightarrow 1110\ 1011$$

Sau đó cộng thêm 1

$$1110\ 1011 + 1 = 1110\ 1100$$

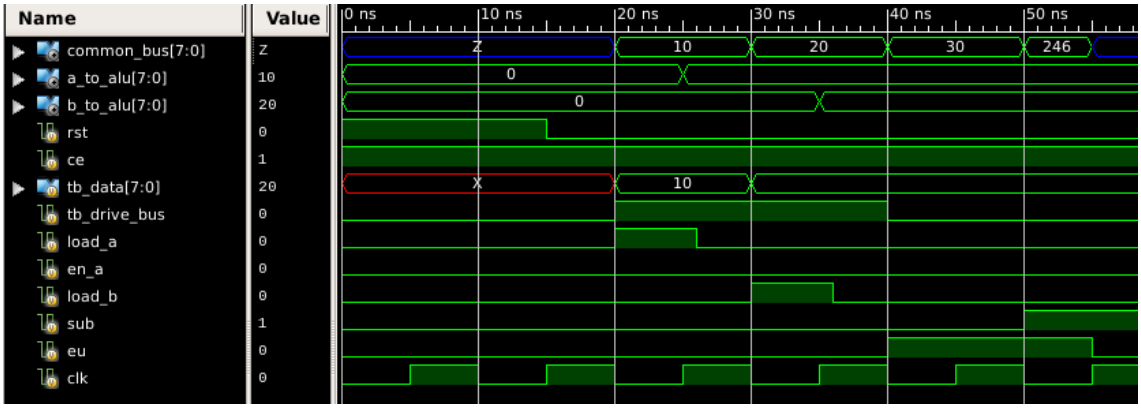
Thực hiện phép cộng 10 + (-20)

$$0000\ 1010 + 1110\ 1100 = 1111\ 0110\ (246dec)$$

4.2 Kết quả chạy của các cụm module

Chia thành 2 nhóm để chạy testbench kiểm thử: nhóm các khối điều khiển lệnh và nhóm các khối tính toán.

4.2.1 Nhóm các khối tính toán



Hình 4.8 Waveform cụm module tính toán

Nhóm các khối tính toán bao gồm các module: ALU, Register A và B, mục đích là để kiểm tra khả năng kết nối của từng module, kiểm tra dữ liệu có hoạt động trôi chảy và nhịp nhàng, có bị dính phải ghi đè hay xung đột dữ liệu. Sau khi kiểm tra nhóm thấy, cụm module hoạt động đúng với yêu cầu logic

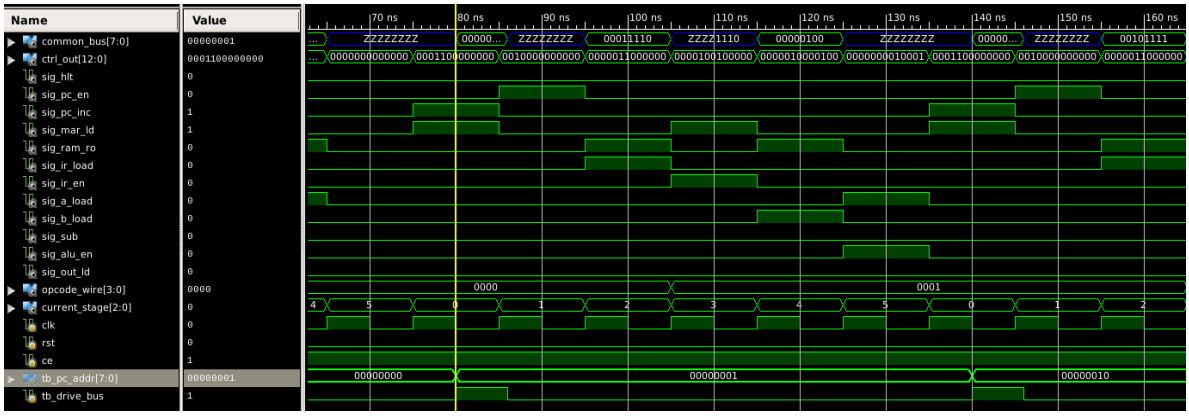
Nạp vào A: trên bus sẽ xuất hiện các giá trị 8-bit, đầu tiên là giá trị 10dec, khi đó lệnh load_a được nạp và giá trị 10dec sẽ được nạp vào thanh ghi a.

Nạp vào B: trên bus xuất hiện giá trị 20, load_b được kích lên mức cao load_b = 1, giá trị 20 được tải vào thanh ghi b

Tính phép cộng: sau khi cả A và B đều chứa giá trị, thì ngay lập tức tính toán mà không cần chờ lệnh điều khiển từ controller (ALU mặc định tính phép cộng). Sau khi cộng xong Eu = 1 để cho phép xuất giá trị ngược ra bus

Tính phép trừ: lúc này giá trị A = 10 và B = 20, khi lệnh sub = 1 thì thực hiện A – B = 10 – 20 = 246 (trừ bù 2)

4.2.2 Nhóm các khối điều khiển



Hình 4.9 Waveform cụm module điều khiển

Nhóm các khối điều khiển bao gồm các module: PC, IR và Controller, để kiểm tra module có hoạt động đúng hay không, sẽ dựa vào kiểm tra các state của lệnh opcode. Kiểm tra các 12-bit tín hiệu của controller có được phát đúng theo từng state đã được thiết kế không. Ví dụ: nhóm lấy opcode = 0001 để kiểm tra.

State 0: PC xuất giá trị, giá trị từ PC đi qua bus rồi vào MAR. Bus lúc này mang giá trị 0000 0001, nhưng sau khi hết S0 thì bus về rỗng



State 1: PC đếm lên, pc_inc được kích



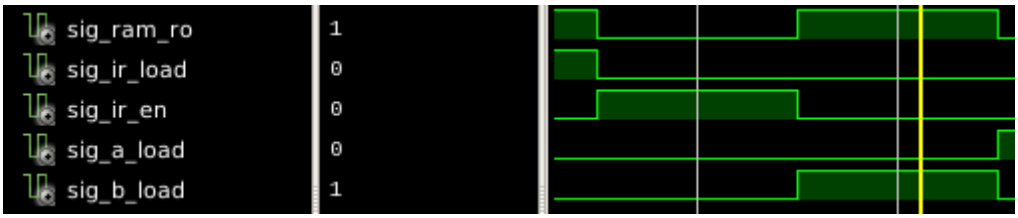
State 2: Ram xuất dữ liệu ra ngoài, IR lấy dữ liệu vào, giá trị bus hiện tại Bus = 0001 1110



State 3: đưa opcode xuống Controller, đưa oprand ra bus, bus = zzzz 1110



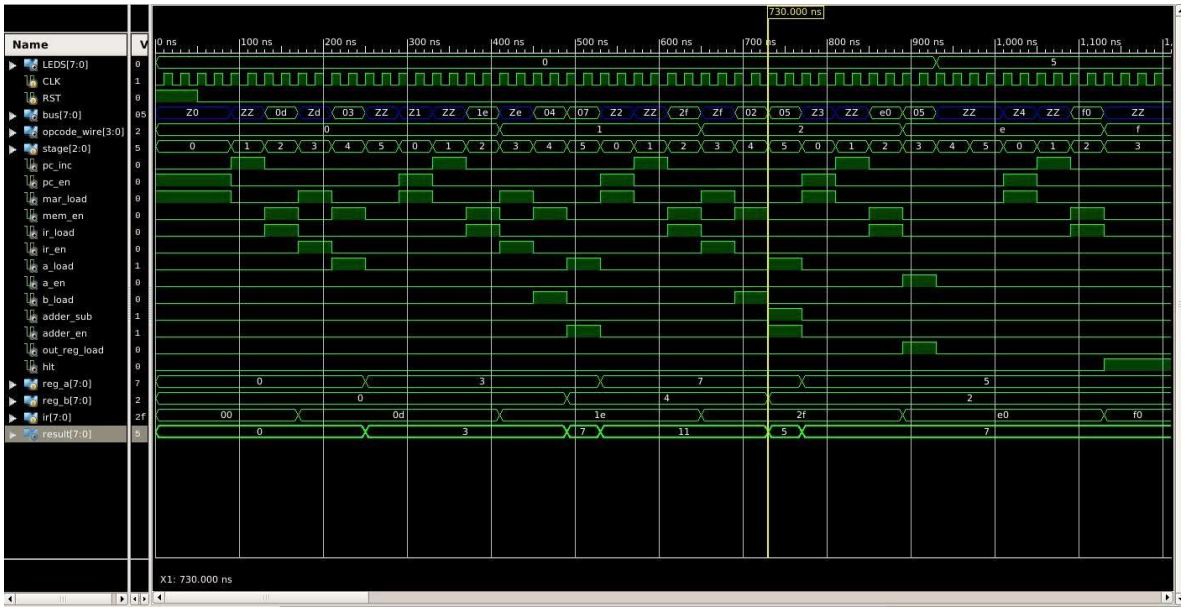
State 4: RAM xuất dữ liệu ra bus, thanh ghi B lấy dữ liệu



State 5: ALU xuất giá trị sau khi tính toán và nạp vào lại thanh ghi A



4.2.3 Kết quả tổng quan chung của hệ thống



Hình 4.10 Waveform tổng quan

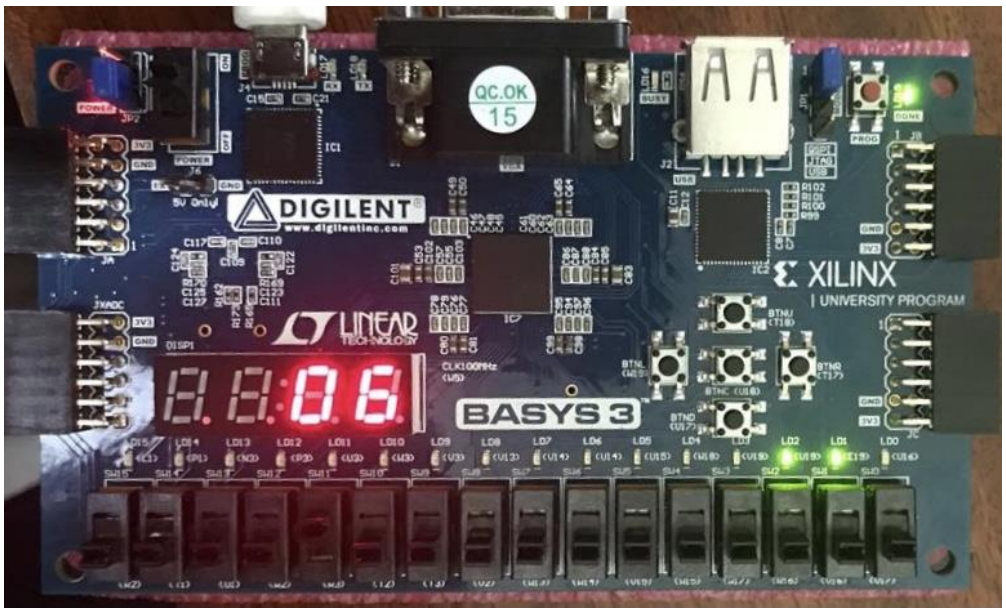
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Sau quá trình viết code và xem xét các dạng waveform, kết quả của từng module chạy chính xác như thiết kế ban đầu đã đề ra, các cụm module hoạt động nhịp nhàng có trình tự và chính xác.

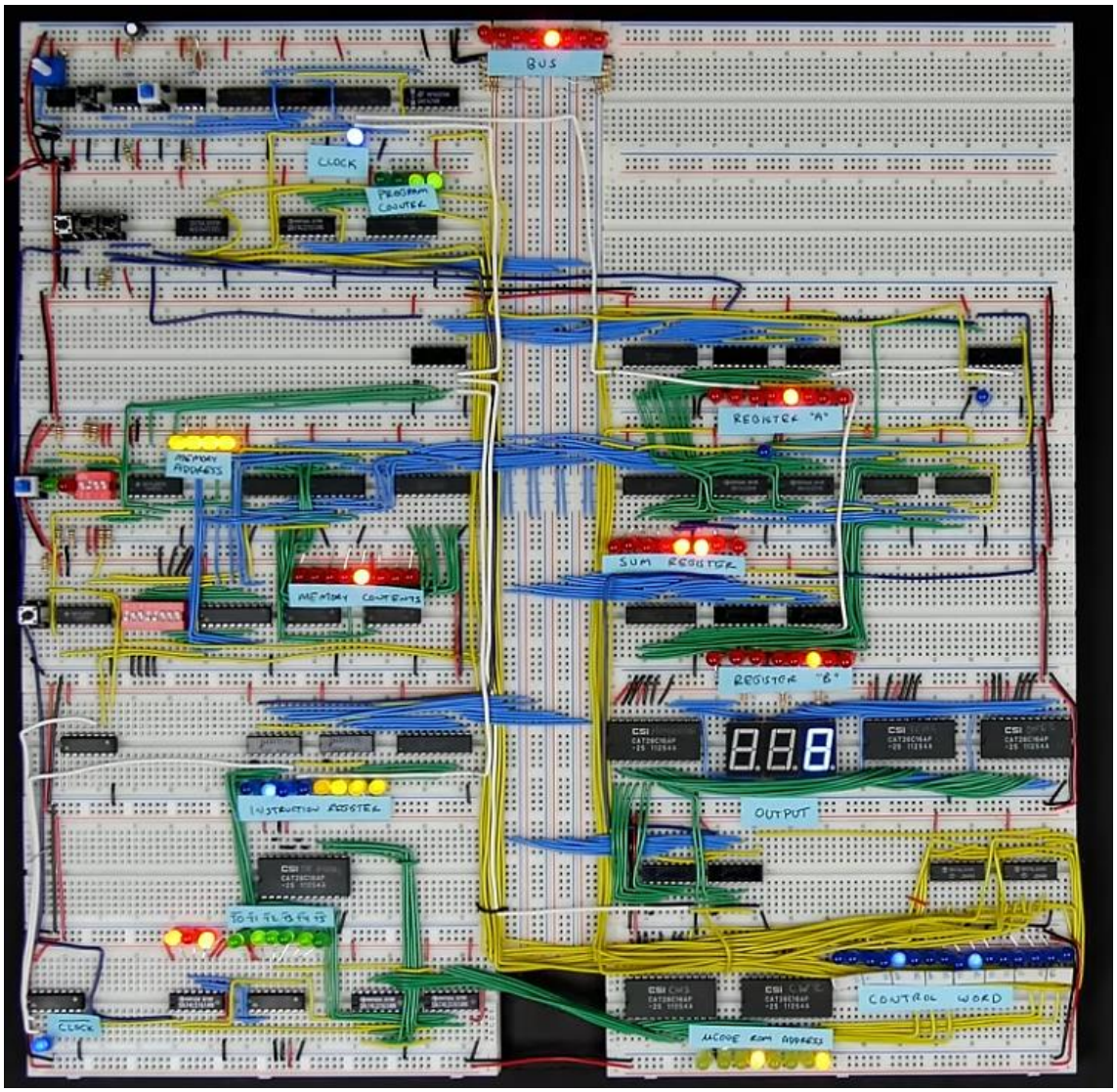
Tuy nhiên, kết quả chỉ dừng lại trên mô phỏng phần mềm, nhóm vẫn chưa đổ code vào board/kit nên có thể phát sinh lỗi ngoài mong muốn.

Nhóm sẽ lấy nguồn tham khảo trên internet ảnh kết quả sau khi đổ board (nguồn: karenok.git), sau khi chạy kết quả được chứa trong RAM, màn hình led sẽ hiện ra kết quả cuối cùng



Hình 5.1 Kết quả sau khi chạy (nguồn: karenok.git)

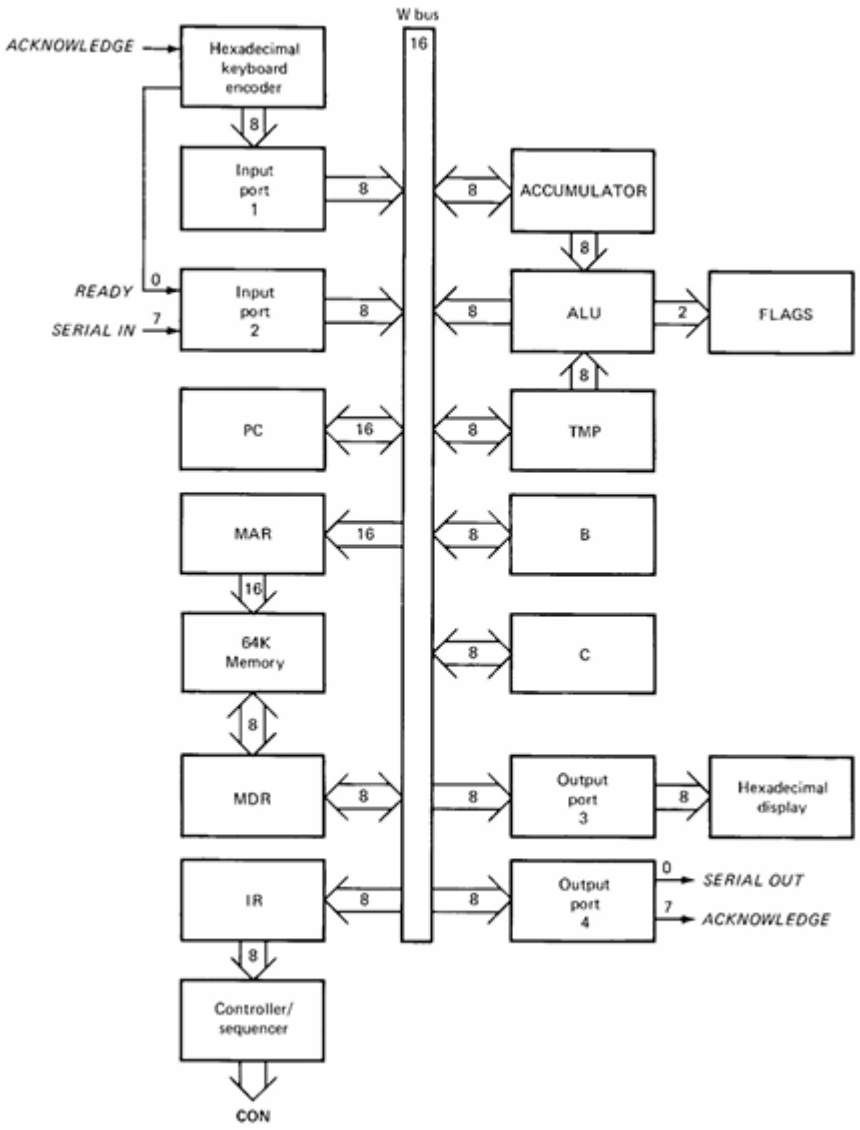
Tham khảo thêm về máy tính SAP-1 được dựng từ các linh kiện phần cứng



Hình 5.2 Build SAP-1 from scrap – Ben Eater

5.2 Hướng phát triển

Việc phát triển từ SAP-1 đến SAP-2 chính là một cuộc "đại nhảy vọt", tiến hóa từ một máy tính thô sơ mang giá trị giáo dục đến một vi xử lý thực tiễn như Intel 8080, 8085



Hình 5.3 Kiến trúc của SAP-2

Một số tính năng, nâng cấp vượt trội hơn so với phiên bản đầu:

PC rộng bộ đếm chương trình lên đến 16-bits, đếm từ 0000 đến FFFF

Bộ nhớ lớn: SAP-2 có địa chỉ cho 64KB, gồm 2KB ROM và còn lại là RAM, rất lớn nếu so với 16byte của SAP-1

Tập lệnh lớn: SAP-1 chỉ có 5 lệnh, ở phiên bản thứ 2 số lượng tập lệnh đã tăng lên 42-bits

Bổ sung thêm thanh ghi cờ

Mở rộng các cổng đầu vào, đầu ra.

TÀI LIỆU THAM KHẢO

Digital Computer Electronics, Third Edition – Albert Paul Malvino, Phd Jerald A.Brown

Sách Digital Electronics bản online: https://drive.google.com/file/d/1g9VK7DQuiCMUlins-J77bPicDu4BEXPz/view (Slide 146)

WEB mô phỏng vận hành: https://sap1-simulator.almeda.io/

Giải thích lý thuyết kiến trúc: https://karenok.github.io/SAP-1-Computer/

Giải thích điện tử: https://www.youtube.com/playlist?list=PLowKtXNTBypGqImE405J2565dvjafglHU

Quy trình hoạt động: https://www.youtube.com/watch?v=hdiFJS8qQtU https://www.youtube.com/watch?v=TSu9aFIbeB8

Mô phỏng trên logisim: https://www.youtube.com/watch?v=SM5Tq_x2Rc0&list=PLzTuuMapW9aB33OuaKEplyrQBnYdAcfL7

Tham khảo: https://github.com/dangrie158/SAP-1